



Clause-reachability is undecidable in legal contracts

Giorgio Delzanno¹ · Cosimo Laneve² · Arnaud Sangnier¹ · Gianluigi Zavattaro²

Accepted: 20 February 2026 / Published online: 19 March 2026
© The Author(s) 2026

Abstract

Micro-Stipula is a stateful calculus in which clauses can be activated either through interactions with the external environment or by the evaluation of time expressions. Despite the apparent simplicity of its syntax and operational model, the combination of state evolution, time reasoning, and nondeterminism gives rise to significant analytical challenges. In particular, we show that determining whether a clause is never executed is undecidable. We formally prove that this undecidability result holds even for syntactically restricted fragments: namely, the *time-ahead* fragment, where all time expressions are strictly positive, the *instantaneous* fragment, where all time expressions evaluate to zero, and the *determinate* fragment, where the initial states of functions and events are disjoint. On the other hand, we identify a decidable subfragment: at the intersection of the instantaneous and determinate fragments reachability becomes decidable.

Keywords Reachability analysis · Legal contracts · *Stipula* · Minsky machines · Well-structured transition systems

1 Introduction

Micro-Stipula, noted $\mu\textit{Stipula}$, is a basic calculus defining *legal contracts*, namely sets of clauses that are either (a) *parameterless functions*, to be invoked by the external environment (e.g. by the parties involved in the contract), or (b) *events* that are triggered at given times (e.g. actions to be executed as soon as a deadline expires). The calculus has been devised to study the presence of clauses in legal contracts written in *Stipula* [11, 12] that can never be applied because of unreachable circumstances or of wrong time constraints – so-called *unreachable clauses*. In the legal contract domain, removing such clauses when the contract is drawn up is substantial because they might be considered too oppressive by parties and make the legal relationship fail.

While dropping unreachable code is a very common optimization in compiler construction of programming lan-

guages [1, 4], the presence of time expressions in $\mu\textit{Stipula}$ events makes the optimization complex. In particular, when the time expressions are logically inconsistent with the contract behaviour, the corresponding event (and its continuation) becomes unreachable.

In [24] we have defined an *unreachable clauses* analyzer that uses symbolic expressions to approximate time expressions at static-time and that computes an over-approximation of reachable clauses by means of a closure operation based on a fixpoint technique. The analyzer, whose prototype is at [30], is sound (every clause it spots is unreachable) but not complete (there may be unreachable clauses that are not recognized). The definition of a complete algorithm for unreachable clauses was left as an open problem.

In this paper we study the foregoing problem for three fragments of $\mu\textit{Stipula}$ that are used to model distinctive elements of legal contracts:

$\mu\textit{Stipula}^{\text{TA}}$: time expressions are always positive – the corresponding events allow one to define *future obligations* such as the power to exercise an option may only last until a deadline is met. For example, a standard clause in a legal contract for renting a device is

– *If the Borrower returns the device k hours after the rental, she shall pay Euro cost to the Lender. (The amount has been pre-paid at the start of the rental.)*

This clause is transposed in $\mu\textit{Stipula}$ with an event like

✉ G. Delzanno
giorgio.delzanno@unige.it

C. Laneve
cosimo.laneve@unibo.it

A. Sangnier
arnaud.sangnier@unige.it

G. Zavattaro
gianluigi.zavattaro@unibo.it

¹ DIBRIS, University of Genova, Genova, Italy

² DISI, University of Bologna, Bologna, Italy

```

now + k >> @Using {
  cost  $\rightarrow$  Lender // send money to the Lender
}  $\Rightarrow$  @End

```

which is affirming that the money stored in `cost` is sent to the Lender because the Borrower is Using the device when the deadline expires. The expressions `@Using` and `@End` indicate states of the contract.

$\mu\text{Stipula}^I$: time expressions are always `now` – the corresponding events permit to define *judicial enforcements* such as the *immediate* activation of a dispute resolution mechanism by an authority when a party challenges the content or the execution of the contract. For example, a contract for renting a device might also contain a clause for resolution of problems:

- If the Lender detects a problem, he may trigger a resolution process that is managed by the Authority. The Authority will immediately enforce resolution to either the Lender or the Borrower.

In this case, the $\mu\text{Stipula}$ clause is the event

```

now >> @Problem {
  // immediately enforce resolution to Lender
  // or Borrower
}  $\Rightarrow$  @Solved

```

where the *immediate resolution* is implemented by a time expression `now`. In this case, if the Lender and the Borrower have not yet resolved the issue – the contract is in a state `Problem` – then the Authority enforces a resolution by, for example, sending a part of `cost` to Lender and the remaining part to Borrower.

$\mu\text{Stipula}^D$: functions and events do not have initial states in common; events in this fragment allow one to model *exceptional behaviours* that must be performed *before* any party may invoke a function. This type of restriction is quite common in legal contracts, particularly when formalizing a clause that outlines the consequences of a party breaching a condition. Section A contains a legal contract written in $\mu\text{Stipula}^D$.

We demonstrate that, even for the aforesaid fragments of $\mu\text{Stipula}$, there is no complete algorithm for determining unreachable clauses. The proof technique is based on reducing the halting problem for Minsky machines (finite state machines with two registers) [27] to the clause-unreachability problem. The $\mu\text{Stipula}$ encodings of Minsky machines model registers' values with multiplicities of events and extensively use the features that (a) events preempt both the invocation of functions and the progression of time and (b) events that are not executed in the current time slot are garbage-collected when the time progresses. The encoding of $\mu\text{Stipula}^{TA}$ is particularly complex because we need to decouple in two different time slots the events corresponding to the two registers and recreate them when the time progresses.

We then restrict to $\mu\text{Stipula}^{DI}$, a fragment of $\mu\text{Stipula}$ that is the intersection of $\mu\text{Stipula}^I$ and $\mu\text{Stipula}^D$, and

prove that the corresponding contracts are an instance of well-structured transition systems (WSTS) [18], whereby the reachability of clause is decidable as it is an instance of the *coverability* problem (which is decidable in WSTS [18]). To achieve this result, we had to modify the semantics of $\mu\text{Stipula}$ by restricting the application of the time progression to states in which functions can be invoked (thus it is disabled in the states where events are executed). Hereafter, the correspondence between the models using the two different progression rules has been analyzed to demonstrate reachability results for $\mu\text{Stipula}^{DI}$.

The rest of this paper is structured as follows. Section 2 presents the calculus $\mu\text{Stipula}$ with examples and the semantics. Section 3 contains the undecidability results for $\mu\text{Stipula}^{TA}$, $\mu\text{Stipula}^I$ and $\mu\text{Stipula}^D$. Section 4 contains the decidability results about $\mu\text{Stipula}^{DI}$. Section 5 reports and discusses related work and Sect. 6 presents general conclusions and future work. Appendix A highlights the relevance in a legal context of the fragments of $\mu\text{Stipula}$ studied in this paper. The proofs of our propositions, lemmas and theorems are reported in Appendix B.

This paper is a revised and extended version of [14] which includes more explanations of the results and the details of the (un)decidability proofs.

2 The calculus $\mu\text{Stipula}$

$\mu\text{Stipula}$ is a calculus of contracts. A contract is declared by the term

$$\text{stipula } C \{ \text{init } Q \quad F \}$$

where C is the name of the contract, Q is the *initial state* and a F is a sequence of *functions*. We use a set of *states*, ranged over $Q, Q', \dots$; and a set of *function names* $f, g, \dots$. The above contract is defined by the keyword `stipula` and is initially in the state specified by the `init` keyword. The syntax of functions F , events W and time expressions t is the following:

<i>Functions</i>	$F ::= _ \mid @Q f \{ W \} \Rightarrow @Q' F$
<i>Events</i>	$W ::= _ \mid t \gg @Q \Rightarrow @Q' W$
<i>Time expressions</i>	$t ::= \text{now} + k \quad (k \in \text{Nat})$

Contracts transit from one state to another either by invoking a *function* or by running an *event*. Functions $@Q f \{ W \} \Rightarrow @Q' F$ are invoked by the *external environment* and define the state Q when the invocation is admitted and the state Q' when the execution of f terminates.

Events W are sequences of *timed continuations* that are created by functions and schedule a transition in future execution. More precisely, the term $t \gg @Q \Rightarrow @Q' W$ schedules a transition from Q to Q' at t time slot ahead the current time if the contract will be in the state Q . The time expressions are

Table 1 The operational semantics of $\mu\text{Stipula}$

$\frac{\begin{array}{c} \text{[FUNCTION]} \\ @Q \text{ f } \{ W \} \Rightarrow @Q' \in C \quad \Psi' = \text{LC}_{Q \cdot \text{f} \cdot Q'}(W) \\ \Psi, Q \rightarrow \end{array}}{C(Q, _, \Psi) \xrightarrow{\text{f}} C(Q, \Psi' \Rightarrow Q', \Psi)}$	$\frac{\begin{array}{c} \text{[EVENT-MATCH]} \\ \Psi = \emptyset \gg_n Q \Rightarrow Q' \mid \Psi' \end{array}}{C(Q, _, \Psi) \xrightarrow{\text{ev}_n} C(Q, _ \Rightarrow Q', \Psi')}$
$\begin{array}{c} \text{[STATE-CHANGE]} \\ C(Q, \Psi' \Rightarrow Q', \Psi) \longrightarrow C(Q', _, \Psi' \mid \Psi) \end{array}$	$\frac{\begin{array}{c} \text{[TICK]} \\ \Psi, Q \rightarrow \end{array}}{C(Q, _, \Psi) \longrightarrow C(Q, _, \Psi \downarrow)}$

additions $\text{now} + k$, where k is a constant (a natural number representing, for example, *minutes*); now is a place-holder that will be replaced by 0 during the execution, see rule [FUNCTION] in Table 1. We always shorten $\text{now} + 0$ into now .

Restriction and notations We write $@Q \text{ f } \{ W \} \Rightarrow @Q' \in C$ when the function $@Q \text{ f } \{ W \} \Rightarrow @Q'$ is in the contract C . Similarly for events. We assume that *a function is uniquely determined by the tuple* $Q \cdot \text{f} \cdot Q'$, that is the initial and final states and the function name. In the same way, an event is uniquely determined by the tuple $Q \cdot \text{ev}_n \cdot Q'$, where n is the line-code of the event.¹ Functions and events are generically called *clauses* and, since tuples $Q \cdot \text{f} \cdot Q'$ and $Q \cdot \text{ev}_n \cdot Q'$ uniquely identify functions and events, we will also call them clauses and write $Q \cdot \text{f} \cdot Q' \in C$ and $Q \cdot \text{ev}_n \cdot Q' \in C$.

2.1 Examples

As a first, simple example considers the PingPong contract:

```

1 stipula PingPong {
2   init Q0
3   @Q0 ping {
4     now + 1 >> @Q1 => @Q2
5   } => @Q1
6   @Q2 pong {
7     now + 2 >> @Q3 => @Q0
8   } => @Q3
9 }
```

The contract contains two functions: *ping* and *pong*. In particular *ping* may be invoked only if the contract is in the state $Q0$, *pong* when the contract is in $Q2$. Functions (i) make the contract transit in the state specified by the term “ $\Rightarrow @Q$ ” (see lines 5 and 8) and (ii) make the events in their body to be scheduled. In particular, an event $\text{now} + k \gg @Q \Rightarrow @Q'$ (see lines 4 and 7) is a timed continuation that can run when the time is k time slots ahead to the clock value when the

function is called and the state of the contract is Q . The only effect of executing an event is the change of the state. When no event can be executed in a state either *the time progresses* (a tick occurs) or *a function is invoked*. The progression of time does not modify a state.

In the PingPong contract, the initial state is $Q0$ where only *ping* may be invoked; no event is present because they are created by executing functions. The invocation of *ping* makes the contract transit to $Q1$ and creates the event at line 4, noted ev_4 . In $Q1$ there is still a unique possibility: executing ev_4 . However, to execute it, it is necessary to wait 1 minute (assuming that the clock period is 1 minute) due to the time expression $\text{now} + 1$. Then the state becomes $Q2$ indicating that *pong* may be invoked, thus letting the contract transit to $Q3$ where, after 2 minutes (the expression $\text{now} + 2$), the event at line 7 can be executed and the contract returns to $Q0$. In PingPong, every clause is reachable.

The following Sample contract has an event that is unreachable:

```

1 stipula Sample {
2   init Init
3   @Init f {
4     now + 0 >> @Go => @End
5   } => @Run
6   @Init g { } => @Go
7 }
```

Let us discuss the issue. Sample has two functions at lines 3 and 6, called *f* and *g*, respectively. The two functions may be invoked in *Init*, however the invocation of one of them excludes the other because their final states are not *Init*. Therefore the event at line 4, which is inside *f*, is unreachable since it can run only if *g* is executed.

2.2 The operational semantics

The meaning of $\mu\text{Stipula}$ primitives is defined operationally by a transition relation between configurations. A configuration, ranged over by $C, C', \dots$, is a tuple $C(Q, \Sigma, \Psi)$ where

- C is the contract name;

¹ We assume the code of $\mu\text{Stipula}$ contracts to be organized in lines of code, and each line contains at most one event definition.

- Q is the current state of the contract;
- Σ is either $_$ or a term $\Psi \Rightarrow Q$. Σ represents either an empty body (hence, a clause can be executed or the time may progress) or a continuation where a set of events Ψ must be evaluated;
- Ψ is a (possibly empty) multiset of *pending events* that have been already scheduled for future execution but not yet triggered. In particular, Ψ is either $_$, when there are no pending events, or it is $k_1 \gg_{n_1} Q_1 \Rightarrow Q'_1 \mid \dots \mid k_h \gg_{n_h} Q_h \Rightarrow Q'_h$ where “ $\mid$ ” is commutative and associative with identity $_$. In every term $k_i \gg_{n_i} Q_i \Rightarrow Q'_i$, the constant k_i is obtained from the time expression t_i of the corresponding event by dropping *now*. The index n_i is the *line-code* of the event.

The function that turns a *sequence of events* $\text{now} + k \gg @Q \Rightarrow @Q'$ into a *multiset of terms* $k \gg_n Q \Rightarrow Q'$ is $\text{LC}_{Q, \mathcal{F}, Q'}(W)$ (see rule [FUNCTION], the trivial definition of this function is omitted). This function also drops the “ $@$ ” from the states.

The transition relation of $\mu\text{Stipula}$ is $\mathbb{C} \xrightarrow{\mu} \mathbb{C}'$, where μ is either *empty* $_$ or $\mathcal{F}$ or ev_n (the label ev_n indicates the event at line n). The formal definition of $\mathbb{C} \xrightarrow{\mu} \mathbb{C}'$ is given in Table 1 using

- the predicate $\Psi, Q \rightarrow$, whose definition is

$$\Psi, Q \rightarrow \stackrel{\text{def}}{=} \begin{cases} \text{true} & \text{if } \Psi = _ \\ \text{false} & \text{if } \Psi = 0 \gg_n Q \Rightarrow Q' \mid \Psi' \\ \Psi', Q \rightarrow & \text{if } \Psi = k \gg_n Q' \Rightarrow Q'' \mid \Psi' \\ & \text{and } (k \neq 0 \text{ or } Q' \neq Q) \end{cases}$$

- the function $\Psi \downarrow$, whose definition is

$$\begin{aligned} (\Psi \mid \Psi') \downarrow &= \Psi \downarrow \mid \Psi' \downarrow \\ (k + 1 \gg_n Q' \Rightarrow Q'') \downarrow &= k \gg_n Q' \Rightarrow Q'' \\ (0 \gg_n Q' \Rightarrow Q'') \downarrow &= _ \end{aligned}$$

A discussion about the four rules follows. Rule [FUNCTION] defines invocations: the label specifies the function name $\mathcal{F}$. The transition may occur provided (i) the contract is in the state Q that admits invocations of $\mathcal{F}$ and (ii) no event can be triggered – *cf.* the premise $\Psi, Q \rightarrow$ (event’s execution preempts function invocation). Rule [STATE-CHANGE] says that a contract changes state by adding the sequence of events W to the multiset of pending events once *now* has been dropped from time expressions. Rule [EVENT-MATCH] specifies that an event handler may run provided Σ is $_$, the time guard of the event has value 0 and the initial state of the event is the same of the contract’s state. Rule [TICK] defines the progression of time. This happens when the contract has an empty Σ and no event can be triggered. In this case, the events with time value 0 are garbage-collected and the time values of the other events are decreased by one. The rules [FUNCTION] and [STATE-CHANGE] might have been squeezed in one rule only. We have preferred to keep them

apart for compatibility with *Stipula* (where functions’ and events’ bodies may also contain statements).

The *initial configuration* of a $\mu\text{Stipula}$ contract

$$\text{stipula } C \{ \text{init } Q \quad \mathcal{F} \}$$

is $\mathbb{C}_{\text{init}} = C(Q, _, _)$; the *set of configurations* of C are denoted by C_C .

We write $\mathbb{C} \rightarrow \mathbb{C}'$ if there is a μ (as said above, μ may be either $_$ or a function name or an event) such that $\mathbb{C} \xrightarrow{\mu} \mathbb{C}'$. We also write $\mathbb{C} \rightarrow^* \mathbb{C}'$, called *computation*, if there are $\mu_1, \dots, \mu_h$ such that $\mathbb{C} \xrightarrow{\mu_1} \dots \xrightarrow{\mu_h} \mathbb{C}'$. Labels are useful in the examples (and proofs) to highlight the clauses that are executed. However, while in [24], they were introduced to ease the formal reasonings, labels may be overlooked in this work. Let $\text{TS}(C) = (C_C, \rightarrow)$ be the transition system associated to C .

Remark 1

Few issues about the semantics in Table 1 are worth to be remarked.

- $\mu\text{Stipula}$ has three *causes for nondeterminism*: (i) two distinct functions can be invoked in a state, (ii) either a function is invoked or the time progresses (in this case the function may be invoked at a later time), and (iii) if two events may be executed at the same time, then one is chosen nondeterministically and is executed.
- Rule [TICK] defines the progression of time. This happens when the contract has no event to trigger. Henceforth, the complete execution of a function or of an event cannot last more than a single time unit. It is worth to notice that this semantics admits the paradoxical phenomenon that an endless sequence of function invocations does not make time progress (*cf.* the encoding of the Minsky machines in $\mu\text{Stipula}^T$). This paradoxical behaviour, which is also present in process calculi with time [21, 28], might be removed by adjusting the semantics so to progress time when a maximal number of functions has been invoked. To ease the formal arguments we have preferred to stick to the simpler semantics.
- The semantics of $\mu\text{Stipula}$ in this paper is different from, yet equivalent to, [12, 24]. In the literature, configurations have clock values that are incremented by the tick-rule. Then, in the [FUNCTION] rule, the variable *now* is replaced by the current clock value (and not dropped, as in our rule). We have chosen the current presentation because it eases the reasoning about expressivity.

To illustrate $\mu\text{Stipula}$ semantics, we discuss the computations of the *PingPong* contract. Consider $\text{Ev}_4 = 1 \gg_4 Q_1 \Rightarrow Q_2$ and $\text{Ev}_7 = 2 \gg_7 Q_3 \Rightarrow Q_0$. We write $\text{Ev}_i^{(-j)}$ to indicate the Ev_i where the time guard has been decreased by j (time) units.

The contract may initially perform a number of [TICK] transitions, say k , and then a [FUNCTION] one. Therefore we have (on the right we write the rule that is used):

$$\begin{array}{lcl} \text{PingPong}(Q0, _, _) & & \\ \xrightarrow{k} \text{PingPong}(Q0, _, _) & [\text{TICK}] & \\ \xrightarrow{\text{ping}} \text{PingPong}(Q0, \text{Ev}_4 \triangleright Q1, _) & [\text{FUNCTION}] & \\ \xrightarrow{} \text{PingPong}(Q1, _, \text{Ev}_4) & [\text{STATE-CHANGE}] & \\ \xrightarrow{} \text{PingPong}(Q1, _, \text{Ev}_4^{(-1)}) & [\text{TICK}] & \\ \xrightarrow{\text{ev}_4} \text{PingPong}(Q1, _ \triangleright Q2, _) & [\text{EVENT-MATCH}] & \\ \xrightarrow{} \text{PingPong}(Q2, _, _) & [\text{STATE-CHANGE}] & \end{array}$$

In $\text{PingPong}(Q2, _, _)$, the contract may perform a [FUNCTION] executing `pong`. Then the computation continues as follows:

$$\begin{array}{lcl} \xrightarrow{\text{pong}} \text{PingPong}(Q2, \text{Ev}_7 \triangleright Q3, _) & [\text{FUNCTION}] & \\ \xrightarrow{} \text{PingPong}(Q3, _, \text{Ev}_7) & [\text{STATE-CHANGE}] & \\ \xrightarrow{2} \text{PingPong}(Q3, _, \text{Ev}_7^{(-2)}) & [\text{TICK}] & \\ \xrightarrow{\text{ev}_7} \text{PingPong}(Q3, _ \triangleright Q0, _) & [\text{EVENT-MATCH}] & \\ \xrightarrow{} \text{PingPong}(Q0, _, _) & [\text{STATE-CHANGE}] & \end{array}$$

Definition 1 (State reachability)

Let C be a $\mu\text{Stipula}$ contract with initial configuration C_{init} . A state Q is reachable in C if and only if there exists a configuration $C(Q, _, \Psi)$ such that $C_{\text{init}} \xrightarrow{*} C(Q, _, \Psi)$.

Our notion of *state reachability* closely relates to several well-established properties in the verification literature. In particular, it aligns with the concept of *control state reachability*, first introduced by Alur and Dill in the context of timed automata [2]. There, the problem is defined as determining whether, given an automaton A and a control state q , there exists a run of A that reaches q . Control state reachability has been extensively studied across various computational models. For instance, it has been examined in the setting of lossy Minsky machines [25], as well as in lossy FIFO channel systems [5, 10]. In the framework of Petri nets, a related problem is the *coverability of a target marking*, as formulated in [23].

As discussed in the Introduction, a corresponding and central property in the context of $\mu\text{Stipula}$ is what we refer to as *clause reachability* as defined below.

Definition 2 (Clause reachability)

Let C be a $\mu\text{Stipula}$ contract with initial configuration C_{init} . A clause $Q \cdot f \cdot Q'$ (or $Q \cdot \text{ev}_n \cdot Q'$) in C is reachable if there exists a computation $C \xrightarrow{*} C' \xrightarrow{\mu} C''$ with $\mu = f$ (or $\mu = \text{ev}_n$).

Clause reachability and state reachability are closely related in the fragments of $\mu\text{Stipula}$ considered in this paper. Their equivalence is established in the next subsection.

2.3 Relevant sublanguages

We will consider the following fragments of $\mu\text{Stipula}$ whose relevance has been already discussed in the Introduction:

$\mu\text{Stipula}^I$, called instantaneous $\mu\text{Stipula}$, is the fragment where the time expression of all the events is always `now + 0`;

$\mu\text{Stipula}^{TA}$, called time-ahead $\mu\text{Stipula}$, is the fragment where every time expression of the events is `now + k`, with $k > 0$;

$\mu\text{Stipula}^D$, called determinate $\mu\text{Stipula}$, is the fragment where the sets of initial states of functions and of events have empty intersection; *i.e.*, for each function $@Q \cdot f \{ W \} \Rightarrow @Q'$ and event $t \gg @Q'' \Rightarrow @Q'''$ in a contract, we impose $Q \neq Q''$;

$\mu\text{Stipula}^{DI}$, called determinate-instantaneous $\mu\text{Stipula}$, is the intersection between $\mu\text{Stipula}^D$ and $\mu\text{Stipula}^I$; *i.e.*, for each function $@Q \cdot f \{ W \} \Rightarrow @Q'$ and event $t \gg @Q'' \Rightarrow @Q'''$ in a contract, we impose $Q \neq Q''$ and $t = \text{now} + 0$.

The focus of this paper is about the (un)decidability of clause reachability in the above fragments, because our ultimate goal is to investigate the possibility to implement a sound and complete unreachable-clause analyser. We now prove that clause reachability is decidable if and only if state reachability is decidable, for all the considered fragments. This will allow us to resort to state reachability that, as previously commented, is a more standard property in the literature.

Theorem 1

Clause reachability is decidable in $\mu\text{Stipula}^I$ (respectively, $\mu\text{Stipula}^{TA}$, $\mu\text{Stipula}^D$ and $\mu\text{Stipula}^{DI}$) if and only if state reachability is decidable in $\mu\text{Stipula}^I$ (respectively, $\mu\text{Stipula}^{TA}$, $\mu\text{Stipula}^D$ and $\mu\text{Stipula}^{DI}$).

Proof

We simply show that, in all the considered fragments, each of the two problems can be reduced to the other one.

Consider a contract C , expressed in any of the above four fragments of $\mu\text{Stipula}$, and one of its clauses. Consider now a modified contract C' obtained from C by modifying the considered clause in such a way that its effect is that of entering a new fresh state Q' (non present in C). Such contract C' belongs to the same fragment of C and, moreover, we have that the considered clause is reachable in C if and only if the state Q' is reachable in C' .

Consider now a contract C , expressed in any of the above four fragments of $\mu\text{Stipula}$, and one of its states Q . It is not restrictive to consider Q different from the initial state, otherwise state reachability becomes a trivial property that always holds. Consider now the set S of clauses in C whose effect is that of entering in state Q . We have that Q is reachable in C if and only if at least one of the clauses in S is reachable. Notice that the set S is finite, hence checking the reachability of one clause in S corresponds to the conjunction of finitely many instances of the clause reachability problem. $\square$

Table 2 The $\mu\text{Stipula}^I$ contract modelling a Minsky machine

Let M be a Minsky machine with initial state Q_0 . Let I_M be the $\mu\text{Stipula}^I$ contract

stipula I_M { init Start F_M }

where F_M contains the functions

– @Start fstart { now $\gg$ @a Q_0 $\Rightarrow$ @b Q_0 } $\Rightarrow$ @ Q_0

– for every instruction $Q : \text{Inc}(R_i, Q')$, with $i \in \{1, 2\}$:

@Q fincQ { now $\gg$ @dec $_i$ $\Rightarrow$ @ackdec $_i$

now $\gg$ @bQ $\Rightarrow$ @Q'

now $\gg$ @aQ' $\Rightarrow$ @bQ'

} $\Rightarrow$ @aQ

– for every instruction $Q : \text{DecJump}(R_i, Q', Q'')$, with $i \in \{1, 2\}$:

@Q fdecQ { now $\gg$ @ackdec $_i$ $\Rightarrow$ @aQ

now $\gg$ @bQ $\Rightarrow$ @Q''

now $\gg$ @aQ'' $\Rightarrow$ @bQ''

} $\Rightarrow$ @dec $_i$

@Q fzeroQ { now $\gg$ @zero $_i$ $\Rightarrow$ @aQ

now $\gg$ @bQ $\Rightarrow$ @Q'

now $\gg$ @aQ' $\Rightarrow$ @bQ'

} $\Rightarrow$ @dec $_i$

– @dec $_1$ fdec1 { } $\Rightarrow$ @zero $_1$ and @dec $_2$ fdec2 { } $\Rightarrow$ @zero $_2$

3 Undecidability results

To show the undecidability of state reachability, we rely on a reduction technique from a Turing-complete computational model to $\mu\text{Stipula}^I$, $\mu\text{Stipula}^{TA}$, and $\mu\text{Stipula}^D$. As Turing-powerful formalism we consider Minsky machines [27]. A Minsky machine is an automaton with two registers R_1 and R_2 holding arbitrary large natural numbers, a finite set of states $Q, Q', \dots$, and a program P consisting of a finite sequence of numbered instructions of the following type:

- $Q : \text{Inc}(R_i, Q')$: in the state Q , increment R_i and go to the state Q' ;
- $Q : \text{DecJump}(R_i, Q', Q'')$: in the state Q , if the content of R_i is zero then go to the state Q' , else decrease R_i by 1 and go to the state Q'' .

A configuration of a Minsky machine is given by a tuple (Q, v_1, v_2) where Q indicates the state of the machine and v_1 and v_2 are the contents of the two registers. A transition of a Minsky machine is denoted by $\rightarrow_M$. We assume that the machine has an *initial state* Q_0 and a *final state* Q_F that has no instruction starting at it. The *halting problem* of a Minsky machine is assessing whether there exist v_1, v_2 such that (Q_F, v_1, v_2) is reachable starting from $(Q_0, 0, 0)$. This problem is undecidable [27]. In the rest of the section, we will demonstrate the undecidability of state reachability for $\mu\text{Stipula}^I$, $\mu\text{Stipula}^{TA}$, and $\mu\text{Stipula}^D$ by providing encodings of Minsky machines. The encodings we use are increasingly complex. Therefore, we will discuss the undecidability results starting from the simplest one. Here, we present the Minsky machines encodings and informally discuss their correctness while the details of the proofs can be found in Appendix B.

3.1 Undecidability for $\mu\text{Stipula}^I$

Table 2 defines the encoding of a Minsky machine M into a $\mu\text{Stipula}^I$ contract I_M . The relevant invariant of the encoding is that, every time M transits to (Q, v_1, v_2) then I_M may transit to $I_M(Q, _, \Psi)$, where the number of events

$$0 \gg \text{@dec}_1 \Rightarrow \text{@ackdec}_1 \text{ and } 0 \gg \text{@dec}_2 \Rightarrow \text{@ackdec}_2$$

in Ψ are v_1 and v_2 , respectively. Additionally, a transition $(Q, v_1, v_2) \rightarrow_M (Q', v'_1, v'_2)$ corresponds to a sequence of transitions $I_M(Q, _, \Psi) \xrightarrow{*} I_M(Q', _, \Psi')$ with either (i) a fincQ function, if the Minsky machine performs an *Inc* instruction, or (ii) either a fdecQ or a fzeroQ function, if the Minsky machine performs a *DecJump* instruction. In particular, the function fincQ has the ability to add one instance of the event $0 \gg \text{@dec}_1 \Rightarrow \text{@ackdec}_1$ (or $0 \gg \text{@dec}_2 \Rightarrow \text{@ackdec}_2$). The function fdecQ has the effect of consuming one instance of the event $0 \gg \text{@dec}_1 \Rightarrow \text{@ackdec}_1$ (resp. $0 \gg \text{@dec}_2 \Rightarrow \text{@ackdec}_2$), by entering the state @dec $_1$ (resp. @dec $_2$) which triggers such event. Also the function fzeroQ enters in one of the states @dec $_i$, but in this case the computation will have the ability to continue only if the state @zero $_i$ will be reached (this because the produced event $0 \gg \text{@zero}_i \Rightarrow \text{@aQ}$ must be triggered to allow the computation to continue). But @zero $_i$ can be reached only if no event $0 \gg \text{@dec}_i$ is present, because only in this case the function fdec $_i$ can be invoked (remember that events have priority w.r.t. function invocations).

We finally observe that the transitions $I_M(Q, _, \Psi) \xrightarrow{*} I_M(Q', _, \Psi')$ which mimic the Minsky machine step $(Q, v_1, v_2) \rightarrow_M (Q', v'_1, v'_2)$ are not the unique possible transitions. Nevertheless, in case different alternative transitions

Let M be a Minsky machine with initial state Q_0 . Let TA_M be the $\mu\text{Stipula}^{\text{TA}}$ contract

stipula $TA_M \{ \text{init } Q_0 \quad F_M \}$
 with F_M containing the functions
 – for every instruction $Q: \text{Inc}(R_i, Q')$, with $i \in \{1, 2\}$:
 @Q fincQ { now + 1 $\gg$ @dec_{*i*} $\Rightarrow$ @ackdec_{*i*}
 now + 1 $\gg$ @Q' $\Rightarrow$ @end
 } $\Rightarrow$ @Q'
 – for every instruction $Q: \text{DecJump}(R_i, Q', Q'')$, with $i \in \{1, 2\}$:
 @Q fddecQ { now + 1 $\gg$ @ackdec_{*i*} $\Rightarrow$ @nextQ''_{*i*}
 now + 1 $\gg$ @wait $\Rightarrow$ @dec_{*i*}
 now + 2 $\gg$ @dec_{*i*} $\Rightarrow$ @end
 now + 2 $\gg$ @dec_{*i*} $\Rightarrow$ @end
 now + 2 $\gg$ @nextQ''_{*i*} $\Rightarrow$ @end
 now + 2 $\gg$ @ackdec_{*i*} $\Rightarrow$ @end
 now + 2 $\gg$ @ackdec_{*i*} $\Rightarrow$ @end
 now + 3 $\gg$ @Q'' $\Rightarrow$ @end
 } $\Rightarrow$ @wait
 @Q fzeroQ { now + 1 $\gg$ @ackdec_{*i*} $\Rightarrow$ @end
 now + 2 $\gg$ @next $\Rightarrow$ @Q'
 now + 1 $\gg$ @wait $\Rightarrow$ @dec_{*i*}
 now + 3 $\gg$ @Q' $\Rightarrow$ @end
 } $\Rightarrow$ @wait
 – for $i \in \{1, 2\}$ and for every state Q of M , we have the following functions:
 @wait fwait { } $\Rightarrow$ @end
 @dec_{*i*} fdec_{*i*} { } $\Rightarrow$ @dec_{*i*}
 @dec_{*i*} fdec_{*i*} { } $\Rightarrow$ @next
 @ackdec_{*i*} fackdec_{*i*} { now + 2 $\gg$ @dec_{*i*} $\Rightarrow$ @ackdec_{*i*} } $\Rightarrow$ @dec_{*i*}
 @nextQ_{*i*} fnextQ_{*i*} { now + 1 $\gg$ @next $\Rightarrow$ @Q } $\Rightarrow$ @dec_{*i*}

states” occurring at odd time clocks. For this reason we add erroneous transitions at even time clocks from management states to `end` (a state without outgoing transitions) and at odd time clocks from machine states to `end`. Similarly to Table 2, a unit in the register i is encoded by an event `now + 1` $\Rightarrow$ `deci` $\Rightarrow$ `ackdeci` (assuming to be in a machine state). Therefore the instruction $Q : Inc(R_i, Q')$, which occurs in a machine state Q , amounts to adding to the next time-clock such event and the erroneous event `now + 1` $\Rightarrow$ `Q'` $\Rightarrow$ `end` that makes the contract transit to `end` if it is still in Q' at the beginning of the next time-clock.

The encoding of $Q : DecJump(R_i, Q', Q'')$ is more convoluted because we have to move all the events $now + 1 \gg @dec_i \Rightarrow @ackdec_i$ ahead two clock units (except one, if the corresponding register value is positive). Assume an invocation of $fdecQ$ occurs and $i = 1$. Then a bunch of events is created (see the body of $fdecQ$ in Table 3) and the contract transits into $wait$. In this state, a $[TICK]$ transition can occur; hence the time values of the events are decreased by one. Then $0 \gg wait \Rightarrow dec_1$ is enabled and the protocol responsible for moving ahead all the events $0 \gg dec_1 \Rightarrow ackdec_1$ (except one) and all the events $0 \gg dec_2 \Rightarrow ackdec_2$ starts. The protocol works as follows:

1. since the state is $\text{dec}_1, 0 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1$ is fired (assume the value of R_1 is positive) and the contract transits to ackdec_1 ;

State reachability is undecidable for the fragment $\mu\text{Stipula}^I$.

Also in this case we reduce from the halting problem of Minsky machines to state reachability in $\mu\textit{Stipula}^{\text{TA}}$. The encoding of a machine M is defined in Table 3. In this case, states of the $\mu\textit{Stipula}^{\text{TA}}$ contract alternate between “*machine states*” occurring, say, at even time clocks, and “*management*

2. in $\text{ackdec}_1, 0 \gg \text{ackdec}_1 \Rightarrow \text{nextQ}''_1$ is fired and the contract transits to state nextQ''_1 (one of the events $0 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1$ has been erased without being moved ahead);
3. in nextQ''_1 , the function

$$\begin{aligned} & @\text{nextQ}''_1 \text{ fnextQ}''_1 \{ \text{now} + 1 \gg @\text{next} \Rightarrow @Q'' \} \\ & \Rightarrow @\text{dec}_1 \end{aligned}$$

can be invoked. A transition to dec_1 happens and the event $1 \gg \text{next} \Rightarrow Q''$ is created. When the transfer protocol terminates, this event will make the contract transit to Q'' ;

4. at this stage the transfer of events $0 \gg \text{dec}_i \Rightarrow \text{ackdec}_i$ occurs. At first the protocol moves all the events $0 \gg \text{dec}_i \Rightarrow \text{ackdec}_i$ (every such event is fired and then the function fackdec_i that recreates the same event at $\text{now} + 2$ is executed) then the function fdec_i is invoked and the same protocol is applied to the events $0 \gg \text{dec}_2 \Rightarrow \text{ackdec}_2$;
5. at the end of the transfers, the function fdec_2 is invoked and the contract transits to next ;
6. in next , no event nor function can be executed, therefore a $[\text{TICK}]$ occurs and the time values of the events are decreased by one (in particular those $2 \gg \text{dec}_i \Rightarrow \text{ackdec}_i$ that were transferred at step 4). Then $0 \gg \text{next} \Rightarrow Q''$ that was created at step 3 is executed and the contract transits to Q'' .

When the register R_1 is 0 and fdecQ is invoked then the event at step 2 cannot be produced and the computation is fated to reach an end state (by fdec_1 , fdec_2 or by $[\text{TICK}]$ and then performing $0 \gg \text{dec}_i \Rightarrow \text{end}$). If, on the contrary, the invoked function is fzeroQ , the same protocol as above is used to transfer the events $0 \gg \text{dec}_i \Rightarrow \text{ackdec}_i$ (in this case with $i = 2$ only) and the contract reaches the step 5 where, after a $[\text{TICK}]$, the event $\text{now} + 2 \gg \text{next} \Rightarrow Q'$ can be executed. The undecidability result for $\mu\text{Stipula}^{\text{TA}}$ follows.

Theorem 3

State reachability is undecidable for the fragment $\mu\text{Stipula}^{\text{TA}}$.

3.3 Undecidability for $\mu\text{Stipula}^{\text{D}}$

Also in this case we reduce from the halting problem of Minsky machines to state reachability in $\mu\text{Stipula}^{\text{D}}$. In $\mu\text{Stipula}^{\text{D}}$, functions and events start in different states. Therefore the encoding of Table 3 is inadequate since we used the expedient that events preempt functions when enabled in the same state to make the contract transit to the end state (which indicates an error). For $\mu\text{Stipula}^{\text{D}}$ we need to refine the sequence *machine-management states* in order to have extra management over erroneous operations (decrease of zero register or zero-test of a positive register). The idea is to manage at different times the events $\text{dec}_1 \Rightarrow \text{ackdec}_1$ and

$\text{dec}_2 \Rightarrow \text{ackdec}_2$ that model registers' units. In particular, if the contract is in a (machine) state Q at time 0 then $\text{dec}_1 \Rightarrow \text{ackdec}_1$ are at time 1 and $\text{dec}_2 \Rightarrow \text{ackdec}_2$ are at time 3. The states at times 2 and 4 perform management operations. Therefore the sequence of states becomes

$$\begin{aligned} & \text{machine-state} \rightarrow \text{transfer1-state} \rightarrow \text{management1-state} \\ & \rightarrow \text{transfer2-state} \rightarrow \text{management2-state} \end{aligned}$$

where every state is one tick ahead the previous one. The states *transfer1-state* and *transfer2-state* manage the transfer of all the events $\text{dec}_1 \Rightarrow \text{ackdec}_1$ and of the events $\text{dec}_2 \Rightarrow \text{ackdec}_2$ ahead five clock units.

Clearly, misplaced $[\text{TICK}]$ transitions may break the rigidity of the protocol. This means that it is necessary to stop the simulation if a wrong $[\text{TICK}]$ transition is performed. We already used a similar mechanism in Table 2. In that case, a management event at time 0 (that is created in the past transition) is necessary to simulate the Minsky machine transition; in turn, the simulation creates a similar management event for the next one. Henceforth, if a tick occurs before the invocation of a function (thus erasing registers' values that were events at time 0, as well) then the simulation stops because the management event is also erased.

A similar expedient cannot be used for the $\mu\text{Stipula}^{\text{D}}$ encoding because the registers' values are at different times (+1 and +3 with respect to the machine state) and erasing the management event with a tick may be useless if the $\mu\text{Stipula}^{\text{D}}$ function produces an equal management event, which is the case when the corresponding transition is circular (initial and final states are the same). Therefore we refine the technique in Table 2 by adding sibling functions and the simulation uses standard functions or sibling ones according to the presence of the management event $0 \gg \text{notickA} \Rightarrow \text{cont}$ or of $0 \gg \text{notickB} \Rightarrow \text{cont}$. For example, the encoding of $Q : \text{Inc}(R_1, Q')$ is (the events of register R_1 are at $\text{now} + 1$):

```
@Q: fAincQ {
  now + 1 >> @dec1 => @ackdec1
  now >> @cont => @Q'
  now >> @notickB => @cont
}>> @notickA

@Q: fBincQ {
  now + 1 >> @dec1 => @ackdec1
  now >> @cont => @Q'
  now >> @notickA => @cont
}>> @notickB
```

Assuming to be in a configuration with state Q and event $0 \gg \text{notickA} \Rightarrow \text{cont}$, the unique function that can be invoked is fAincQ ; thereafter, the combined effect of $0 \gg \text{notickA} \Rightarrow \text{cont}$ and $0 \gg \text{cont} \Rightarrow Q'$ allows the contract

Table 4 The $\mu\text{Stipula}^D$ contract modelling a Minsky machine

Let M be a Minsky machine with initial state Q_0 . Let D_M be the $\mu\text{Stipula}^D$ contract

$$\text{stipula } D_M \{ \text{init Start } F_M \}$$

with F_M contains the functions

– @Start fstart { now $\gg$ @notickA $\Rightarrow$ @cont } $\Rightarrow$ @Q₀

– for every $Q : \text{Inc}(R_1, Q')$:

@Q fAincQ { now + 1 $\gg$ @dec₁ $\Rightarrow$ @ackdec₁ now $\gg$ @cont $\Rightarrow$ @Q' now $\gg$ @notickB $\Rightarrow$ @cont } $\Rightarrow$ @notickA	@Q fBincQ { now + 1 $\gg$ @dec₁ $\Rightarrow$ @ackdec₁ now $\gg$ @cont $\Rightarrow$ @Q' now $\gg$ @notickA $\Rightarrow$ @cont } $\Rightarrow$ @notickB
--	--

– for every $Q : \text{Inc}(R_2, Q')$:

@Q fAincQ { now + 3 $\gg$ @dec₂ $\Rightarrow$ @ackdec₂ now $\gg$ @cont $\Rightarrow$ @Q' now $\gg$ @notickB $\Rightarrow$ @cont } $\Rightarrow$ @notickA	@Q fBincQ { now + 3 $\gg$ @dec₂ $\Rightarrow$ @ackdec₂ now $\gg$ @cont $\Rightarrow$ @Q' now $\gg$ @notickA $\Rightarrow$ @cont } $\Rightarrow$ @notickB
--	--

– for every $Q : \text{DecJump}(R_1, Q', Q'')$:

@Q fAdecQ { now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @Q''_start1 now + 1 $\gg$ @s1notick $\Rightarrow$ @cont now $\gg$ @cont $\Rightarrow$ @dec₁ } $\Rightarrow$ @notickA @Q fAzeroQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 3 $\gg$ @ackdec₂ $\Rightarrow$ @copy₂ now + 3 $\gg$ @c2notickA $\Rightarrow$ @cont now + 5 $\gg$ @dec₂ $\Rightarrow$ @Q' now + 5 $\gg$ @notickB $\Rightarrow$ @cont } $\Rightarrow$ @notickA	@Q fBdecQ { now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @Q''_start1 now + 1 $\gg$ @s1notick $\Rightarrow$ @cont now $\gg$ @cont $\Rightarrow$ @dec₁ } $\Rightarrow$ @notickB @Q fBzeroQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 3 $\gg$ @ackdec₂ $\Rightarrow$ @copy₂ now + 3 $\gg$ @c2notickA $\Rightarrow$ @cont now + 5 $\gg$ @dec₂ $\Rightarrow$ @Q' now + 5 $\gg$ @notickA $\Rightarrow$ @cont } $\Rightarrow$ @notickB
--	--

– for every $Q : \text{DecJump}(R_2, Q', Q'')$:

@Q fAdecQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @copy₁ now + 1 $\gg$ @c1notickA $\Rightarrow$ @cont now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 3 $\gg$ @ackdec₂ $\Rightarrow$ @Q''_start2 now + 3 $\gg$ @s2notick $\Rightarrow$ @cont } $\Rightarrow$ @notickA @Q fAzeroQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @copy₁ now + 1 $\gg$ @c1notickA $\Rightarrow$ @cont now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 5 $\gg$ @dec₂ $\Rightarrow$ @Q' now + 5 $\gg$ @notickB $\Rightarrow$ @cont } $\Rightarrow$ @notickA	@Q fBdecQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @copy₁ now + 1 $\gg$ @c1notickA $\Rightarrow$ @cont now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 3 $\gg$ @ackdec₂ $\Rightarrow$ @Q''_start2 now + 3 $\gg$ @s2notick $\Rightarrow$ @cont } $\Rightarrow$ @notickB @Q fBzeroQ { now $\gg$ @cont $\Rightarrow$ @dec₁ now + 1 $\gg$ @ackdec₁ $\Rightarrow$ @copy₁ now + 1 $\gg$ @c1notickA $\Rightarrow$ @cont now + 2 $\gg$ @dec₁ $\Rightarrow$ @dec₂ now + 5 $\gg$ @dec₂ $\Rightarrow$ @Q' now + 5 $\gg$ @notickA $\Rightarrow$ @cont } $\Rightarrow$ @notickB
--	--

– the management functions in Table 5.

Table 5 The management functions of Table 4 (Q is a state of the Minsky machine)

<pre> @Q_start1 fQstart1 { now >> @cont => @dec₁ now >> @ackdec₁ => @copy₁ now >> @c1notickA => @cont now + 1 >> @dec₁ => @dec₂ now + 2 >> @ackdec₂ => @copy₂ now + 2 >> @c2notickA => @cont now + 4 >> @dec₂ => @Q now + 4 >> @notickA => @cont } => @s1notick </pre>			
<pre> @Q_start2 fQstart2 { now >> @cont => @dec₂ now >> @ackdec₂ => @copy₂ now >> @c2notickA => @cont now + 2 >> @dec₂ => @Q now + 2 >> @notickA => @cont } => @s2notick </pre>			
<pre> @copy₁ fAcopy1 { now >> @cont => @dec₁ now >> @ackdec₁ => @copy₁ now >> @c1notickB => @cont now + 5 >> @dec₁ => @ackdec₁ } => @c1notickA </pre>	<pre> @copy₂ fAcopy2 { now >> @cont => @dec₂ now >> @ackdec₂ => @copy₂ now >> @c2notickB => @cont now + 5 >> @dec₂ => @ackdec₂ } => @c2notickA </pre>	<pre> @copy₁ fBcopy1 { now >> @cont => @dec₁ now >> @ackdec₁ => @copy₁ now >> @c1notickA => @cont now + 5 >> @dec₁ => @ackdec₁ } => @c1notickB </pre>	<pre> @copy₂ fBcopy2 { now >> @cont => @dec₂ now >> @ackdec₂ => @copy₂ now >> @c2notickA => @cont now + 5 >> @dec₂ => @ackdec₂ } => @c2notickB </pre>

to transit to Q' with an additional event $1 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1$ (corresponding to a register increment) and the presence of $0 \gg \text{notickB} \Rightarrow \text{cont}$ that compels the next instruction, if any, to be a sibling one (e.g. fBincQ').

Tables 4 and 5 define the encoding of a Minsky machine M into a $\mu\text{Stipula}^D$ contract D_M . The reader may notice that the management functions fQstart1 and fQstart2 in Table 5 do not have sibling functions because they always produce a management event $k \gg \text{notickA} \Rightarrow \text{cont}$ (with k be either 2 or 4). Actually this is an optimization: they are invoked because a decrement occurred and, in such cases it is not possible that the foregoing management events are used in the simulation of the current instruction. The undecidability result for $\mu\text{Stipula}^D$ follows.

Theorem 4

State reachability is undecidable for the fragment $\mu\text{Stipula}^D$.

4 Decidability for $\mu\text{Stipula}^{DI}$

We demonstrate that state reachability is decidable for $\mu\text{Stipula}^{DI}$. As for the previous section, here we present our proof technique and informally discuss its correctness, while the details of the proofs can be found in Appendix B.

We first recall that, in $\mu\text{Stipula}^{DI}$, for every pair of clauses $Q \cdot \text{f} \cdot Q'$ and $Q'' \cdot \text{ev} \cdot Q'''$ in C , we have $Q \neq Q''$. In our decidability proof, we resort to a variant of the semantics of $\mu\text{Stipula}^{DI}$ with an alternative $[\text{Tick}]$ rule. Let $\text{InitEv}(C)$ be the set of initial states of events in C , where C is a $\mu\text{Stipula}$ contract. Let

$$\frac{[\text{Tick-Plus}]}{Q \notin \text{InitEv}(C)} \\ C(Q, _, \Psi) \longrightarrow C(Q, _, \Psi \downarrow)$$

That is, unlike $[\text{Tick}]$, $[\text{Tick-Plus}]$ may only be used in states that are not initial states of events. Let $\mu\text{Stipula}_+^{DI}$ be the language whose operational semantics uses $[\text{Tick-Plus}]$ instead of $[\text{Tick}]$; we denote with $\longrightarrow_{\text{tp}}$ the transition relation of $\mu\text{Stipula}_+^{DI}$. We observe that, syntactically, nothing is changed: every $\mu\text{Stipula}^{DI}$ contract is a $\mu\text{Stipula}_+^{DI}$ contract and conversely. We denote by $\mathbb{T}\mathbb{S}_{\text{tp}}(C) = (C_C, \longrightarrow_{\text{tp}})$ the transitions system associated to contract C using $\longrightarrow_{\text{tp}}$ as transition rule.

Definition 3

Let $\mathbb{C}$ be a possible configuration of a $\mu\text{Stipula}$ contract. We say that $\mathbb{C}$ is *stuck* if, for every computation $\mathbb{C} \longrightarrow^* \mathbb{C}'$ the transitions therein are always instances of $[\text{Tick}]$.

Proposition 1

Let $C(Q, \Sigma, \Psi)$ be a configuration of a $\mu\text{Stipula}^{DI}$ contract C (or a $\mu\text{Stipula}_+^{DI}$ contract). Then

- (i) whenever $Q \notin \text{InitEv}(C)$ or $\Sigma \neq _:$

$$C(Q, \Sigma, \Psi) \longrightarrow \mathbb{C} \text{ if and only if } C(Q, \Sigma, \Psi) \longrightarrow_{\text{tp}} \mathbb{C};$$

- (ii) whenever $Q \in \text{InitEv}(C)$ and $\Psi = \mathbf{0} \gg_n Q \Rightarrow Q' \mid \Psi'$:

$$C(Q, _, \Psi) \longrightarrow \mathbb{C} \text{ if and only if } C(Q, _, \Psi) \longrightarrow_{\text{tp}} \mathbb{C};$$

- (iii) whenever $Q \in \text{InitEv}(C)$ and $\Psi, Q \nrightarrow$:

$$C(Q, _, \Psi) \text{ is stuck if and only if } C(Q, _, \Psi) \nrightarrow_{\text{tp}}.$$

A consequence of Proposition 1 is that a state Q is reachable in $\mu\text{Stipula}^{\text{DI}}$ if and only if it is reachable in $\mu\text{Stipula}_+^{\text{DI}}$. This allows us to safely reduce state reachability arguments to $\mu\text{Stipula}_+^{\text{DI}}$. In particular we demonstrate that $\text{TS}_{\text{tp}}(C)$, where C is a $\mu\text{Stipula}_+^{\text{DI}}$ contract, is a well-structured transition system.

We begin with some background on well-structured transition systems [18]. A relation $\leq \subseteq X \times X$ is called *quasi-ordering* if it is reflexive and transitive. A *well-quasi-ordering* is a quasi-ordering $\leq \subseteq X \times X$ such that, for every infinite sequence $x_1, x_2, x_3, \dots$, there exist $i < j$ with $x_i \leq x_j$.

Definition 4

A *well-structured transition system* is a tuple $(C, \longrightarrow, \preceq)$ where $(C, \longrightarrow)$ is a transition system and $\preceq \subseteq C \times C$ is a quasi-ordering such that:

- (1) $\preceq$ is a well-quasi-ordering
- (2) $\preceq$ is upward compatible with $\longrightarrow$, i.e., for every $C_1, C'_1, C_2 \in C$ such that $C_1 \preceq C'_1$ and $C_1 \longrightarrow C_2$ there exists C'_2 in C verifying $C'_1 \longrightarrow^* C'_2$ and $C_2 \preceq C'_2$

Given a configuration C of a well-structured transition system, $\text{Pred}(C)$ denotes the set of immediate predecessors of C (i.e., $\text{Pred}(C) = \{C' \mid C' \longrightarrow C\}$) while $\uparrow C$ denotes the set of configurations greater than C (i.e., $\uparrow C = \{C' \mid C \preceq C'\}$). With abuse of notation, we will denote with $\text{Pred}(\cdot)$ and $\uparrow \cdot$ also their natural extension to sets of configurations. A *basis* of an upward-closed set of configurations $\mathcal{D} \subseteq C$ is a set $\mathcal{D}^b$ such that $\mathcal{D} = \bigcup_{C \in \mathcal{D}^b} \uparrow C$. We know that every upward-closed set of a well-quasi-ordering admits a finite basis [18].

Several properties are decidable for well-structured transition systems (under some conditions discussed below) [8, 18], we will consider the following one.

Definition 5

Let $(C, \longrightarrow, \preceq)$ be a well-structured transition system. The *coverability problem* is to decide, given the initial configuration $C_{\text{init}} \in C$ and a target configuration $C \in C$, whether there exists a configuration $C' \in C$ such that $C \preceq C'$ and $C_{\text{init}} \longrightarrow^* C'$.

In well-structured transition systems the coverability problem is decidable when the transition relation $\longrightarrow$, the ordering $\preceq$ and, for every $C \in C$, a finite-basis for $\uparrow \text{Pred}(\uparrow C)$ are effectively computable.

Let us now define the relation $\preceq$ as the least quasi-ordering relation such that $\Psi \preceq \Psi' \mid \Psi'$ for every Ψ' . The relation $\preceq$ is lifted to configurations as follows

$$C(Q, \Sigma, \Psi) \preceq C(Q, \Sigma, \Psi') \quad \text{if} \quad \Psi \preceq \Psi'.$$

It is worth to observe that, according to the relation $\preceq$, the state reachability problem for Q is equivalent to the coverability problem for the initial configuration C_{init} and the target configuration $C(Q, -, -)$.

Lemma 1

For a $\mu\text{Stipula}_+^{\text{DI}}$ contract C , $(C_C, \longrightarrow_{\text{tp}}, \preceq)$ is a well-structured transition system.

It is worth to notice that Lemma 1 does not hold for $\mu\text{Stipula}^{\text{DI}}$ because $\longrightarrow$ is not upward compatible with $\preceq$. In fact, while $C(Q, -, -) \longrightarrow C(Q, -, -)$ with a rule $[\text{Tick}]$ and $C(Q, -, -) \preceq C(Q, -, 0 \gg_n Q \Rightarrow Q')$, the unique computation of $C(Q, -, 0 \gg_n Q \Rightarrow Q')$ when $Q' \in \text{InitEv}(C)$ is (we recall that, in $\mu\text{Stipula}$, events preempt function invocations and ticks):

$$C(Q, -, 0 \gg_n Q \Rightarrow Q') \longrightarrow C(Q, - \Rightarrow Q', -) \longrightarrow C(Q', -, -)$$

and then it gets stuck. Therefore no configuration C is reachable such that $C(Q, -, -) \preceq C$.

Lemma 2

Let $(C, \longrightarrow_{\text{tp}}, \preceq)$ be the well-structured transition system of a $\mu\text{Stipula}_+^{\text{DI}}$ contract. Then, $\longrightarrow_{\text{tp}}$ and $\preceq$ are decidable and there exists an algorithm for computing a finite basis of $\uparrow \text{Pred}(\uparrow C)$ for any $C \in C$.

From Lemma 2 and the above mentioned results, on well-structured transition systems we get the following result.

Theorem 5

The state reachability problem is decidable for the fragment $\mu\text{Stipula}_+^{\text{DI}}$.

As a direct consequence of Proposition 1 and Theorem 5 we have:

Corollary 1

The state reachability problem is decidable for the fragment $\mu\text{Stipula}^{\text{DI}}$.

We conclude with a few remarks on the $\mu\text{Stipula}^{\text{DI}}$ fragment. Although certain legal contracts can indeed be modeled within $\mu\text{Stipula}^{\text{DI}}$ – for example, the contract shown in Listing 2 in Appendix A – this fragment is inherently less expressive than the others. In particular, $\mu\text{Stipula}^{\text{DI}}$ cannot capture scenarios involving future events, such as the modeling of mortgage installment payments, where events are scheduled to occur at future time points. Likewise, deadline constraints cannot be expressed: if a deadline is reached before the function terminating the rental has been invoked, it is necessary that an event originating from the same state as that function triggers a compensation behavior. The restrictions imposed on $\mu\text{Stipula}^{\text{DI}}$ have been deliberately designed so that Lemma 2 holds, while ensuring that the expressiveness of the fragment is not excessively compromised.

5 Related work

The decidability of problems about infinite-state systems has been largely addressed in the literature. We refer to [8] for an overview of the research area.

It turns out that critical features of $\mu\text{Stipula}$, such as garbage-collecting elapsed events or preempting events with respect to functions and progression of time may be modelled by variants of Petri nets with inhibitor and reset arcs. While standard Petri nets, which are infinite-state systems, have decidable problems of reachability, coverability, boundedness, etc. (see, e.g., [17]), the above variants of Petri nets are Turing complete, therefore all non-trivial properties become undecidable [16].

It is not obvious whether Petri nets with inhibitor and reset arcs may be modelled by $\mu\text{Stipula}$ contracts. It seems that these features have different expressive powers than time progression and events. Hence, other formalisms, such as pi-calculus and actor languages, might have a stricter correspondence with $\mu\text{Stipula}$. As regards the decidability of problems in pi-calculus, we recall the decidability of reachability and termination in the depth-bounded fragment of the pi-calculus [26] and the decidability of the reachability problem for various fragments of the asynchronous pi-calculus that feature name generation, name mobility, and unbounded control [3]. Regarding actor languages, in [13], we demonstrated the decidability of termination for stateless actors (actors without fields) and for actors with states when the number of actors is bounded and the state is read-only. It is worth to observe that all these results have been achieved by using techniques that are similar to those used in this paper: either demonstrating that the model of the calculus is a well-structured transition system [18] (for which, under certain computability conditions, the reachability and termination problems are decidable, see Sect. 4) or simulating a Turing complete model, such as the Minsky machines, into the calculus under analysis (hence the undecidability results of problems such as termination).

The $\mu\text{Stipula}$ calculus has some similarities with formal models of timed systems such as timed automata [2]. The control state reachability problem, namely, given a timed automaton A and a control state q , does there exist a run of A that visits q , is known to be decidable [2]. A similar result holds for Timed Networks (TN) [6, 7], a formal model consisting of a family of timed automata with a distinct *controller* defined as a finite-state automaton without clocks. Each process in a TN can communicate with all other processes via rendezvous messages. Control state reachability is also decidable for Timed Networks with transfer actions [9]. A transfer action forces all processes in a given state to move to a successor state as transfer arcs in Petri nets, which allows to move all tokens contained in a certain place to another [19].

$\mu\text{Stipula}$ can also be seen as a language for modelling asynchronous programs in which callbacks are scheduled using timers. Verification problems for formal models of (untimed) asynchronous programs have been considered in [22, 29]. In this context, Boolean program execution is modeled using a pushdown automaton, while asynchronous calls are modeled by adding a multiset of pending callbacks to the global state of the program. Callbacks are only executed when the program stack is empty. Verification of safety properties is decidable for this model via a non-trivial reduction to coverability of Petri nets [20].

6 Conclusions

We have systematically studied the computational power of $\mu\text{Stipula}$, a basic calculus defining legal contracts. The calculus is stateful and features clauses that may be either functions to be invoked by the external environment or events that can be executed at certain time slots. We have demonstrated that in several legally relevant fragments of $\mu\text{Stipula}$ a problem such as reachability of state is undecidable. The decidable fragment, $\mu\text{Stipula}^{\text{DI}}$, is the one whose event and functions start in disjoint states and where events are instantaneous (the time expressions are 0).

We conclude by indicating some relevant line for future research. First of all, the decidability result for $\mu\text{Stipula}^{\text{DI}}$ leaves open the question about the complexity of the state reachability problem in that fragment. Our current conjecture is that the problem is EXPSPACE -complete and we plan to prove this conjecture by reducing the coverability problem for Petri nets into the state reachability problem for $\mu\text{Stipula}^{\text{DI}}$. Another interesting line of research regards the investigation of sound, but incomplete, algorithms for checking state reachability in $\mu\text{Stipula}$. A preliminary algorithm was investigated in [24]. The presented algorithm spots clauses that are unreachable in $\mu\text{Stipula}$ contracts and is not tailored to any particular fragment of the language. The results in this paper show that this algorithm may be improved to achieve completeness when the input contract complies with the $\mu\text{Stipula}^{\text{DI}}$ constraints.

Appendix A: Legal relevance of $\mu\text{Stipula}$ and its fragments

To illustrate the relevance in a legal context of the fragments of $\mu\text{Stipula}$ identified in the paper, we refer to simple legal contracts and employ a subset of operations from the full *Stipula* language. We begin with a legal contract for bike rentals; it consists of three articles:

1. This Agreement shall commence when the Borrower takes possession of the Bike and remains in full force

```

1 stipula Bike_Rental {
2   assets wallet, bike
3   init @Inactive
4   @Inactive Lender : offer[b] {
5     b  $\rightarrow$  bike // the access code of the bike is stored
6               // in the contract
7   }  $\Rightarrow$  @Payment
8   @Payment Borrower : pay[x]
9     (x == cost) {
10      x  $\rightarrow$  wallet // the contract keeps the money
11      bike  $\rightarrow$  Borrower // the Borrower keeps
12               // the bike access code
13      now + k  $\gg$  @Using {
14        wallet  $\rightarrow$  Lender // deadline expired: the
15                       // whole wallet to Lender
16      }  $\Rightarrow$  @End
17    }  $\Rightarrow$  @Using
18    @Using Borrower : end { // bike returned before
19                          // the deadline
20      0.5  $\times$  wallet  $\rightarrow$  wallet, Lender // half wallet
21                               // to Lender
22      wallet  $\rightarrow$  Borrower // half wallet to Borrower
23    }  $\Rightarrow$  @End
24 }

```

Listing 1 The bike-rental contract

and effect until the Bike is returned to Lender. The Borrower shall return the Bike k hours after the rental and will pay Euro cost in advance where half of the amount is of surcharge for late return.

2. Payment. Borrower shall pay the amount specified in Article 1 when this agreement commences.
3. End. If the Bike is returned within the agreed time specified in Article 1, then half of the cost will be returned to Borrower; the other half is given to the Lender. Otherwise the full cost is given to the Lender.

These articles are transposed into $\mu\text{Stipula}$ using a few extensions to the calculus – operations that are almost standard and will be briefly discussed below.

The contract `Bike_Rental` has two *assets*: `wallet` and `bike` that will contain money and the access code of a bike, respectively. The contract starts when the Lender offers an access code b of a bike (line 4) that is stored in the asset `bike` – the operation $b \rightarrow \text{bike}$ at line 5 (this is Article 1 in the legal contract). Then the Borrower can pay – the function at line 8 – which amounts to store in the `wallet` the payment x that has been made (this is Article 2). At the same time the bike code is sent to the Borrower (the operation $\text{bike} \rightarrow \text{Borrower}$ at line 11), so the Borrower can use the bike, and event is scheduled at time $\text{now} + k$ (lines 13–16): if the bike is not returned at that time, the Borrower has to pay the whole amount in the `wallet` as specified in Article 3. The function at lines 18–23 defines the timely return of the bike by the Borrower (Article 3; notice that the operation $0.5 \times \text{wallet} \rightarrow \text{wallet}$, Lender halves the content of the `wallet`; therefore the following operation $\text{wallet} \rightarrow \text{Borrower}$ sends to the Borrower the remaining half).

```

1 stipula alt_Bike_Rental {
2   assets wallet, bike
3   init @Inactive
4   @Inactive Lender : offer[b] {
5     b  $\rightarrow$  bike // the bike access code is stored
6               // in the contract
7   }  $\Rightarrow$  @Payment
8   @Payment Borrower : pay[x]
9     (x  $\geq$  cost) {
10      x  $\rightarrow$  wallet // the contract keeps the money
11      bike  $\rightarrow$  Borrower // the Borrower keeps the bike
12               // access code
13    }  $\Rightarrow$  @Using
14    @Using Borrower : end { // bike returned
15      0.9  $\times$  wallet  $\rightarrow$  wallet, Lender // 90% wallet to
16                               // Lender
17      wallet  $\rightarrow$  Borrower // 10% wallet to Borrower
18    }  $\Rightarrow$  @End
19    @Using Lender : problems(u) {
20      // communicate the problems  $u$  to Authority
21      // that will manage the issue
22    }  $\Rightarrow$  @Manage
23    @Manage Authority : resolution (v) {
24      if (v == 0) {
25        now  $\gg$  @Problem {
26          // immediately enforce resolution to the Lender
27        }  $\Rightarrow$  @PbLender
28      } else { // there is no management to do
29        now  $\gg$  @Problem { // immediately enforce
30                               // resolution to the Borrower
31        }  $\Rightarrow$  @PbBorrower
32      }
33    }  $\Rightarrow$  @Problem

```

Listing 2 The alternative bike-rental contract

We observe that a $\mu\text{Stipula}$ contract can be obtained from Listing 1 by removing all $\rightarrow$ operations, the functions' parameters, and the asset declarations. In particular, the resulting contract belongs to the $\mu\text{Stipula}^{\text{TA}}$ fragment, since k (the renting time) is assumed to be always positive.

As regards the relevance of the other fragments of $\mu\text{Stipula}$, consider the following alternative bike rental contract, in which the Borrower may keep the bike indefinitely, but becomes eligible for a discount on the price if the bike is returned.

1. This Agreement shall commence when the Borrower takes possession of the Bike and remains in full force and effect until the Bike is returned to Lender. The Borrower shall return the Bike at his discretion and will pay Euro cost in advance.
2. Payment. Borrower shall pay the amount cost as specified in Article 1 when this agreement commences.
3. End. If the Bike is returned then 90% of cost will be given to Lender and 10% of cost will be returned to Borrower.
4. Problems and resolutions. If the Lender detects a problem, he may trigger a resolution process that is managed by the Authority. The Authority will immediately enforce resolution to either the Lender or the Borrower.

(To keep things simple, we have omitted the details of Article 4.) The alternative contract is similar to that of Listing 1 up to line 7.

The key lines of Listing 2 are 23–32 where the Authority enforces an immediate resolution in favour of the Lender

or of the Borrower. In these cases, the events are respectively now $\gg \text{@Problem} \{ \} \Rightarrow \text{@PbLender}$ and now $\gg \text{@Problem} \{ \} \Rightarrow \text{@PbBorrower}$. Since the time expressions are always now, the contract belongs to the fragment $\mu\text{Stipula}^I$. Actually, it also belongs to $\mu\text{Stipula}^D$ because the initial state Problem of the two events is different from Inactive, Payment, Using and Manage, which are the initial state of the functions. As such, the contract of Listing 2 fits with the constraints of $\mu\text{Stipula}^{DI}$.

Appendix B: Proofs

B.1 Proofs of Sect. 3

Theorem 2

State reachability is undecidable for the fragment $\mu\text{Stipula}^I$.

Proof

Let M be a Minsky machine with states $\{Q_0, \dots, Q_n\}$ where Q_0 is the initial state. Let I_M be the encoding in $\mu\text{Stipula}^I$ as defined in Table 2 and let $S, S', \dots$ range over states of I_M . The initial state of I_M is $I_M(\text{Start}, -, -)$ and, after a sequence of transitions $[\text{Tick}]$, the contract I_M performs the transitions

$$\begin{aligned} I_M(\text{Start}, -, -) &\xrightarrow{\text{fstart}} I_M(\text{Start}, Ev \Rightarrow Q_0, -) \\ &\longrightarrow I_M(Q_0, -, Ev) \end{aligned}$$

where $Ev = 0 \gg aQ_0 \Rightarrow bQ_0$. Next, let

$$\begin{aligned} \llbracket v_1, v_2 \rrbracket_Q^I &= 0 \gg \underbrace{\text{dec}_1 \Rightarrow \text{ackdec}_1}_{v_1 \text{ times}} \mid 0 \gg \underbrace{\text{dec}_2 \Rightarrow \text{ackdec}_2}_{v_2 \text{ times}} \mid 0 \gg aQ \\ &\Rightarrow bQ. \end{aligned}$$

We demonstrate that

- (1) $(Q_i, v_1, v_2) \longrightarrow_M (Q_j, v'_1, v'_2)$ implies $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^* I_M(Q_j, -, \llbracket v'_1, v'_2 \rrbracket_{Q_j}^I)$
- (2) I_M does not transit to unreachable Minsky's states.
That is, if $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^* I_M(S, \Sigma, \Psi) \longrightarrow I_M(Q_j, \Sigma', \Psi')$ with $S \notin \{Q_0, \dots, Q_n\}$ then $\Sigma' = -$ and $\Psi' = \llbracket v'_1, v'_2 \rrbracket_{Q_j}^I$ and $(Q_i, v_1, v_2) \longrightarrow_M^* (Q_j, v'_1, v'_2)$.

The proof of (1) is a case analysis on the type of Minsky instructions. We omit the simulation of the *Inc* instruction, which is simple, and discuss in detail the simulation of *DecJump*. There are two cases (we assume the register is R_1):

$R_1 > 0$: therefore M performs the transition $(Q_i, v_1, v_2) \longrightarrow_M (Q_j, v_1 - 1, v_2)$. Correspondingly, in I_M , an invocation of

the function fdec_{Q_i} occurs. Then the contract performs the transitions

$$\begin{aligned} I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) &\xrightarrow{\text{fdec}_{Q_i}} I_M(Q_i, E_1|E_2|E_3 \Rightarrow \text{dec}_1, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \\ &\longrightarrow I_M(\text{dec}_1, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I | E_1|E_2|E_3) \end{aligned}$$

where $E_1 = 0 \gg \text{ackdec}_1 \Rightarrow aQ_i$, $E_2 = 0 \gg bQ_i \Rightarrow Q_j$, and $E_3 = 0 \gg aQ_j \Rightarrow bQ_j$. Since $v_1 > 0$, there is at least one event $0 \gg \text{@dec}_1 \Rightarrow \text{ackdec}_1$ in $\llbracket v_1, v_2 \rrbracket_{Q_i}^I$. Then

$$\begin{aligned} I_M(\text{dec}_1, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I | E_1|E_2|E_3) &\longrightarrow I_M(\text{dec}_1, - \Rightarrow \text{ackdec}_1, \llbracket v_1-1, v_2 \rrbracket_{Q_i}^I | E_1|E_2|E_3) \\ &\longrightarrow I_M(\text{ackdec}_1, -, \llbracket v_1-1, v_2 \rrbracket_{Q_i}^I | E_1|E_2|E_3) \\ &\longrightarrow I_M(\text{ackdec}_1, - \Rightarrow aQ_i, \llbracket v_1-1, v_2 \rrbracket_{Q_i}^I | E_2|E_3) \\ &\longrightarrow I_M(aQ_i, -, \llbracket v_1-1, v_2 \rrbracket_{Q_i}^I | E_2|E_3) \\ &\longrightarrow I_M(aQ_i, - \Rightarrow bQ_i, \llbracket v_1-1, v_2 \rrbracket_{Q_j}^I | E_2) \\ &\longrightarrow I_M(bQ_i, -, \llbracket v_1-1, v_2 \rrbracket_{Q_j}^I | E_2) \\ &\longrightarrow I_M(bQ_i, - \Rightarrow Q_j, \llbracket v_1-1, v_2 \rrbracket_{Q_j}^I) \\ &\longrightarrow I_M(Q_j, -, \llbracket v_1-1, v_2 \rrbracket_{Q_j}^I) \end{aligned}$$

$R_1 = 0$: therefore M performs the transition $(Q_i, 0, v_2) \longrightarrow_M (Q_j, 0, v_2)$. Correspondingly, in I_M , an invocation of the function fzero_{Q_i} occurs. Then the contract performs the transitions

$$\begin{aligned} I_M(Q_i, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I) &\xrightarrow{\text{fzero}_{Q_i}} I_M(Q_i, E'_1|E_2|E_3 \Rightarrow \text{dec}_1, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \\ &\longrightarrow I_M(\text{dec}_1, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E'_1|E_2|E_3) \end{aligned}$$

where $E'_1 = 0 \gg \text{zero}_1 \Rightarrow aQ_i$ (E_2 and E_3 are as above). Since $v_1 = 0$, there is no event $0 \gg \text{@dec}_1 \Rightarrow \text{ackdec}_1$ in $\llbracket v_1, v_2 \rrbracket_{Q_i}^I$. Then it is possible to invoke fdec_1 :

$$\begin{aligned} I_M(\text{dec}_1, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E'_1|E_2|E_3) &\xrightarrow{\text{fdec}_1} I_M(\text{dec}_1, - \Rightarrow \text{zero}_1, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E'_1|E_2|E_3) \\ &\longrightarrow I_M(\text{zero}_1, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E'_1|E_2|E_3) \\ &\longrightarrow I_M(\text{zero}_1, - \Rightarrow aQ_i, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E_2|E_3) \\ &\longrightarrow I_M(aQ_i, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | E_2|E_3) \\ &\longrightarrow^* // \text{ similarly to the above case} \\ &\longrightarrow I_M(Q_j, -, \llbracket 0, v_2 \rrbracket_{Q_j}^I) \end{aligned}$$

As regards (2), we assume that $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^* I_M(S, \Sigma, \Psi) \longrightarrow I_M(Q_j, \Sigma', \Psi')$ is such that $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^* I_M(S, \Sigma, \Psi)$ does not contain pairs of transitions $I_M(S'', \Sigma'', \Psi'') \longrightarrow I_M(Q'', \Sigma''', \Psi''')$ with $S'' \notin \{Q_0, \dots, Q_n\}$ and $Q'' \in \{Q_0, \dots, Q_n\}$. The general case follows by recurrence.

The proof is a case analysis on the first transition of $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I)$:

- if $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow I_M(Q_i, -, -)$ is an instance of [TICK]. This case is vacuous: the continuations of $I_M(Q_i, -, -)$ may be either instances of [TICK] or the invocation of exactly one function $f \in \{\text{fincQ}_i, \text{fdecQ}_i, \text{fzeroQ}_i\}$ possibly followed by an invocation of either fdec_1 or fdec_2 and by the execution of the event $0 \gg \text{zero}_1 \Rightarrow \text{aQ}_i$ or $0 \gg \text{zero}_2 \Rightarrow \text{aQ}_i$. Therefore no transition $I_M(S, \Sigma, \Psi) \longrightarrow I_M(Q_j, \Sigma', \Psi')$ will be ever possible.
- if $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \xrightarrow{\text{fincQ}_i} I_M(Q_i, Ev \Rightarrow \text{aQ}_i, \llbracket v_1, v_2 \rrbracket_{Q_i}^I)$ then, according to Table 2, M contains the instruction $Q_i : \text{Inc}(R_1, Q_j)$ or $Q_i : \text{Inc}(R_2, Q_j)$. We consider the first case, i.e., R_1 is incremented (the argument for R_2 is similar). It is easy to verify that $I_M(Q_i, Ev \Rightarrow \text{aQ}_i, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^5 I_M(Q_j, -, \llbracket v_1 + 1, v_2 \rrbracket_{Q_j}^I)$ with a deterministic computation that does not traverse configurations $I_M(S'', \Sigma'', \Psi'')$ with $S'' \in \{Q_0, \dots, Q_n\}$.
- if $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \xrightarrow{\text{fdecQ}_i} I_M(Q_i, Ev \Rightarrow \text{aQ}_i, \llbracket v_1, v_2 \rrbracket_{Q_i}^I)$ then, according to Table 2, M contains $Q_i : \text{DecJump}(R_1, Q', Q'')$ or $Q_i : \text{DecJump}(R_2, Q', Q'')$. We consider the first case, i.e., R_1 is decreased/tested (the argument for R_2 is similar). There are two subcases:
 - $(v_1 > 0)$ In this case it is easy to verify that $I_M(Q_i, Ev \Rightarrow \text{aQ}_i, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^8 I_M(Q'', -, \llbracket v_1 - 1, v_2 \rrbracket_{Q''}^I)$ with a computation that does not traverse configurations $I_M(S'', \Sigma'', \Psi'')$ with $S'' \in \{Q_0, \dots, Q_n\}$. In this case, the computation is not deterministic because the third transition is the invocation of fdec_1 . Actually, it is also possible to perform a transition [TICK]. Analogous to the first case above, after the transition [TICK], no transition $I_M(S, \Sigma, \Psi) \longrightarrow I_M(Q_j, \Sigma', \Psi')$ will be ever possible.
 - $(v_1 = 0)$ This case is vacuous because we have $I_M(Q_i, Ev \Rightarrow \text{dec}_1, \llbracket 0, v_2 \rrbracket_{Q_i}^I) \longrightarrow^2 I_M(\text{dec}_1, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | Ev)$ and, for every $I_M(\text{dec}_1, -, \llbracket 0, v_2 \rrbracket_{Q_i}^I | Ev) \longrightarrow^* I_M(S, \Sigma, \Psi)$, it is easy to verify that $S \notin \{Q_0, \dots, Q_n\}$.
- if $I_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \xrightarrow{\text{fzeroQ}_i} I_M(Q_i, Ev \Rightarrow \text{dec}_1, \llbracket v_1, v_2 \rrbracket_{Q_i}^I)$ then, according to Table 2, M contains $Q_i : \text{DecJump}(R_1, Q', Q'')$ or $Q_i : \text{DecJump}(R_2, Q', Q'')$. We consider the first case, i.e., R_1 is considered (the argument for R_2 is similar). There are two subcases:
 - $(v_1 = 0)$ We have that $I_M(Q_i, Ev \Rightarrow \text{dec}_1, \llbracket v_1, v_2 \rrbracket_{Q_i}^I) \longrightarrow^9 I_M(Q', -, \llbracket 0, v_2 \rrbracket_{Q'}^I)$. As in the case of fdecQ_i , the computation is not deterministic due to the presence of alternative [TICK] transitions, but after a [TICK] no transition $I_M(S, \Sigma, \Psi) \longrightarrow I_M(Q_j, \Sigma', \Psi')$ will be ever possible.
 - $(v_1 > 0)$ Vacuous, analogous to the case of fdecQ_i with $v_1 = 0$.

This concludes the proof. $\square$

Theorem 3

State reachability is undecidable for the fragment $\mu\text{Stipula}^D$.

Proof

Let M be the Minsky machine with states $\{Q_1, \dots, Q_n\}$ and TA_M be the encoding in $\mu\text{Stipula}^{\text{TA}}$ as defined in Table 3. Let

$$\begin{aligned} \llbracket v_1, v_2 \rrbracket_Q^{\text{TA}} &= \underbrace{1 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1}_{v_1 \text{ times}} \mid \underbrace{1 \gg \text{dec}_2 \Rightarrow \text{ackdec}_2}_{v_2 \text{ times}} \mid 1 \gg Q \\ &\Rightarrow \text{end} \end{aligned}$$

and let $\Psi_{\text{TA}}, \Psi'_{\text{TA}}$ be either $_$ or $(i \text{ is either } 1 \text{ or } 2)$

$$\begin{aligned} &0 \gg \text{dec}_1 \Rightarrow \text{end} \mid 0 \gg \text{dec}_2 \Rightarrow \text{end} \mid 0 \gg \text{nextQb}_i \Rightarrow \text{end} \\ &\mid 0 \gg \text{ackdec}_1 \Rightarrow @\text{end} \mid 0 \gg \text{ackdec}_2 \Rightarrow \text{end} \end{aligned}$$

(the cases where Ψ and Ψ' are not $_$ correspond to those where the machine transition is a *DecJump*). We need to prove

(1) $(Q_i, v_1, v_2) \longrightarrow_M (Q_j, v'_1, v'_2)$ implies $\text{TA}_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^{\text{TA}} \mid \Psi) \longrightarrow^* \text{TA}_M(Q_j, -, \llbracket v'_1, v'_2 \rrbracket_{Q_j}^{\text{TA}} \mid \Psi')$

(2) TA_M does not transit to unreachable Minsky's states.

That is, if $\text{TA}_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i} \mid \Psi_{\text{TA}}) \longrightarrow^* \text{TA}_M(S, \Sigma, \Psi)$ with $S \notin \{Q_0, \dots, Q_n\}$ then $\Sigma' = _$ and $\Psi' = \llbracket v'_1, v'_2 \rrbracket_{Q_j}^{\text{TA}} \mid \Psi'_{\text{TA}}$ and also $(Q_i, v_1, v_2) \longrightarrow_M^* (Q_j, v'_1, v'_2)$.

The proofs of (1) and (2) consist of a detailed case analysis on the transitions of TA_M . The proof of (1) is similar to the corresponding one of Theorem 2, therefore omitted. As regards (2), the difference with that of Theorem 2 is the fact that $\text{TA}_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_{Q_i}^{\text{TA}} \mid \Psi_{\text{TA}}) \longrightarrow^* \text{TA}_M(S, \Sigma, \Psi) \longrightarrow \text{TA}_M(Q_j, \Sigma', \Psi')$ contains exactly 3 transitions [TICK] (in the basic case where no state $\{Q_0, \dots, Q_n\}$ is traversed). Otherwise one has to demonstrate that the case becomes vacuous. The details are omitted. In the same way we may conclude the proof. $\square$

Theorem 4

State reachability is undecidable for the fragment $\mu\text{Stipula}^D$.

Proof

Let M be the Minsky machine and D_M be the encoding in $\mu\text{Stipula}^D$ as defined in Tables 4 and 5. The initial state of D_M is $D_M(\text{Start}, -, -)$ and, after a sequence of transitions [TICK], the contract D_M performs the transitions

$$\begin{aligned} D_M(\text{Start}, -, -) &\xrightarrow{\text{fstart}} D_M(\text{Start}, Ev \Rightarrow Q_0, -) \\ &\longrightarrow D_M(Q_0, -, Ev) \end{aligned}$$

where Q_0 is the initial state of M and $Ev = 0 \gg \text{notickA} \Rightarrow \text{cont}$. Next, let

$$\begin{aligned} \llbracket v_1, v_2 \rrbracket_X^D &= \underbrace{1 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1}_{v_1 \text{ times}} \mid \underbrace{3 \gg \text{dec}_2 \Rightarrow \text{ackdec}_2}_{v_2 \text{ times}} \\ &\mid 0 \gg \text{notickX} \Rightarrow \text{cont} \end{aligned}$$

where X may be either A or B . We need to prove

- (1) $(Q_i, v_1, v_2) \longrightarrow_M (Q_j, v'_1, v'_2)$ implies $D_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_X^D) \longrightarrow^* D_M(Q_j, -, \llbracket v'_1, v'_2 \rrbracket_Y^D)$ where X and Y are either A or B ;
- (2) D_M does not transit to unreachable Minsky's states.
That is, if $D_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_X^D) \longrightarrow^* D_M(S, \Sigma, \Psi) \longrightarrow D_M(Q_j, \Sigma', \Psi')$, with $S \notin \{Q_0, \dots, Q_n\}$, then $\Sigma' = -$ and $\Psi' = \llbracket v'_1, v'_2 \rrbracket_Y^D$ and $(Q_i, v_1, v_2) \longrightarrow_M^* (Q_j, v'_1, v'_2)$.

The proof of (1) is a case analysis on the type of Minsky instructions. We omit the simulation of the *Inc* instruction, which is simple, and discuss in detail the simulation of *DecJump*. While the proof is similar to that of Theorem 2, in this case it is much longer because of the complexity of the protocol. In the proof we use the following notation:

$$\{v_1, v_2\}^D = 1 \gg \underbrace{\text{dec}_1 \Rightarrow \text{ackdec}_1}_{v_1 \text{ times}} \mid 3 \gg \underbrace{\text{dec}_2 \Rightarrow \text{ackdec}_2}_{v_2 \text{ times}}$$

that is, $\{v_1, v_2\}^D$ is $\llbracket v_1, v_2 \rrbracket_X^D$ without the event $0 \gg \text{notickX} \Rightarrow \text{cont}$. To highlight $[\text{Tick}]$ transitions we colour them in red.

We assume the involved register is R_1 and that the multiset of events contains $0 \gg \text{notickA} \Rightarrow \text{cont}$ (the case when the multiset contains $0 \gg \text{notickB} \Rightarrow \text{cont}$ is similar). There are two cases:

$R_1 > 0$: therefore M performs the transition $(Q_i, v_1, v_2) \longrightarrow_M (Q_j, v_1 - 1, v_2)$. Correspondingly, in D_M , an invocation of the function fAdecQ_i occurs. Then the contract performs the transitions

$$\begin{aligned} & D_M(Q_i, -, \llbracket v_1, v_2 \rrbracket_A^D) \\ & \xrightarrow{\text{fAdecQ}_i} D_M(Q_i, E_1 | E_2 | E_3 \Rightarrow \text{notickA}, \llbracket v_1, v_2 \rrbracket_A^D) \\ & \longrightarrow D_M(Q_i, - \Rightarrow \text{notickA}, \llbracket v_1, v_2 \rrbracket_A^D | E_1 | E_2 | E_3) \\ & \longrightarrow D_M(\text{notickA}, -, \llbracket v_1, v_2 \rrbracket_A^D | E_1 | E_2 | E_3) \end{aligned}$$

where $E_1 = 1 \gg \text{ackdec}_1 \Rightarrow Q_{j_start1}$, $E_2 = 1 \gg \text{slnotick} \Rightarrow \text{cont}$, and $E_3 = 0 \gg \text{cont} \Rightarrow \text{dec}_1$. Then we have (in the following we omit the labels of the events):

$$\begin{aligned} & D_M(\text{notickA}, -, \llbracket v_1, v_2 \rrbracket_A^D | E_1 | E_2 | E_3) \\ & \longrightarrow D_M(\text{notickA}, - \Rightarrow \text{cont}, \{v_1, v_2\}^D | E_1 | E_2 | E_3) \\ & \longrightarrow D_M(\text{cont}, -, \{v_1, v_2\}^D | E_1 | E_2 | E_3) \\ & \longrightarrow D_M(\text{cont}, - \Rightarrow \text{dec}_1, \{v_1, v_2\}^D | E_2) \\ & \longrightarrow D_M(\text{dec}_1, -, \{v_1, v_2\}^D | E_1 | E_2) \\ & \longrightarrow D_M(\text{dec}_1, -, (\{v_1, v_2\}^D | E_1 | E_2) \downarrow) \end{aligned}$$

where the last transition is an instance of $[\text{Tick}]$; notice that no other transition is possible. Since $v_1 > 0$, there is at

least one event $0 \gg \text{@dec}_1 \Rightarrow \text{ackdec}_1$ in $\{v_1, v_2\}^D \downarrow$. Then

$$\begin{aligned} & D_M(\text{dec}_1, -, (\{v_1, v_2\}^D | E_1 | E_2) \downarrow) \\ & \longrightarrow D_M(\text{dec}_1, - \Rightarrow \text{ackdec}_1, (\{v_1-1, v_2\}^D | E_1 | E_2) \downarrow) \\ & \longrightarrow D_M(\text{ackdec}_1, -, (\{v_1-1, v_2\}^D | E_1 | E_2) \downarrow) \\ & \longrightarrow D_M(\text{ackdec}_1, - \Rightarrow Q_{j_start1}, (\{v_1-1, v_2\}^D | E_2) \downarrow) \\ & \longrightarrow D_M(Q_{j_start1}, -, (\{v_1-1, v_2\}^D | E_2) \downarrow) \end{aligned}$$

At this point, we have to move ahead of 5 time units the events in $\{v_1-1, v_2\}^D \downarrow$. We use the management functions of Table 5, in particular fQ_{j_start1} , then the protocol continues with fAcopy1 alternating with fBcopy1 for v_1-1 times in total, and fAcopy2 alternating with fBcopy2 for v_2 times in total. Let $\prod_{h \in 1..7} E'_h = E'_1 | \dots | E'_7$ and let E'_A , E'_B and V_r^{+k} , where r is either 1 or 2, be defined as follows:

$$\begin{aligned} E'_1 &= \text{@cont} \Rightarrow \text{@dec}_1 \\ E'_2 &= \text{now} \gg \text{@ackdec}_1 \Rightarrow \text{@copy}_1 \\ E'_3 &= \text{now} + 1 \gg \text{@dec}_1 \Rightarrow \text{@dec}_2 \\ E'_4 &= \text{now} + 2 \gg \text{@ackdec}_2 \Rightarrow \text{@copy}_2 \\ E'_5 &= \text{now} + 2 \gg \text{@c2notickA} \Rightarrow \text{@cont} \\ E'_6 &= \text{now} + 4 \gg \text{@dec}_2 \Rightarrow \text{@Q}_j \\ E'_7 &= \text{now} + 4 \gg \text{@notickA} \Rightarrow \text{@cont} \end{aligned}$$

$$\begin{aligned} E'_A &= \text{now} \gg \text{@c1notickA} \Rightarrow \text{@cont} \\ E'_B &= \text{now} \gg \text{@c1notickB} \Rightarrow \text{@cont} \\ V_r^{+k} &= k \gg \text{@dec}_r \Rightarrow \text{@ackdec}_r \end{aligned}$$

We also use the following notation: X may be either A or B . If $X = A$ then X' is B ; if $X = B$ then $X' = A$. Then:

$$\begin{aligned} & D_M(Q_{j_start1}, -, (\{v_1-1, v_2\}^D | E_2) \downarrow) \\ & \xrightarrow{\text{fQ}_{j_start1}} D_M(Q_{j_start1}, \prod_{h \in 1..7} E'_h | E'_A \Rightarrow \text{slnotick}, \\ & \quad (\{v_1-1, v_2\}^D | E_2) \downarrow) \\ & \longrightarrow D_M(Q_{j_start1}, - \Rightarrow \text{slnotick}, \\ & \quad \prod_{h \in 1..7} E'_h | E'_A | (\{v_1-1, v_2\}^D | E_2) \downarrow) \\ & \longrightarrow D_M(\text{slnotick}, -, \prod_{h \in 1..7} E'_h | E'_A | (\{v_1-1, v_2\}^D | E_2) \downarrow) \\ & \longrightarrow D_M(\text{slnotick}, - \Rightarrow \text{cont}, \prod_{h \in 1..7} E'_h | E'_A | \{v_1-1, v_2\}^D \downarrow) \\ & \longrightarrow D_M(\text{cont}, -, \prod_{h \in 1..7} E'_h | E'_A | \{v_1-1, v_2\}^D \downarrow) \\ & \quad // \text{ initially } X = A \text{ and } \ell = 0 \\ & = \left(D_M(\text{cont}, -, \prod_{h \in 1..7} E'_h | E'_X | \{v_1-1, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times}) \right. \\ & \longrightarrow D_M(\text{cont}, - \Rightarrow \text{dec}_1, \\ & \quad \prod_{h \in 2..7} E'_h | E'_X | \{v_1-1, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times}) \\ & \longrightarrow D_M(\text{dec}_1, -, \prod_{h \in 2..7} E'_h | E'_A | \{v_1-1, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times}) \\ & \longrightarrow D_M(\text{dec}_1, - \Rightarrow \text{ackdec}_1, \\ & \quad \prod_{h \in 2..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times}) \\ & \longrightarrow D_M(\text{ackdec}_1, -, \end{aligned}$$

$$\begin{aligned}
 & \prod_{h \in 2..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times} \\
 & \longrightarrow D_M(\text{ackdec}_1, - \Rightarrow \text{copy}_1, \\
 & \quad \prod_{h \in 3..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times} \\
 & \longrightarrow D_M(\text{copy}_1, -, \prod_{h \in 3..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times} \\
 & \xrightarrow{\text{fXcopy}_1} D_M(\text{copy}_1, \prod_{h \in 1..2} E'_h | E'_X | V_1^{+5} \Rightarrow \text{c1notickX}, \\
 & \quad \prod_{h \in 3..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^\ell \text{ times} \\
 & \longrightarrow D_M(\text{copy}_1, - \Rightarrow \text{c1notickX}, \\
 & \quad \prod_{h \in 1..7} E'_h | E'_X | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^{\ell+1} \text{ times} \\
 & \longrightarrow D_M(\text{c1notickX}, -, \\
 & \quad \prod_{h \in 1..7} E'_h | E'_X | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^{\ell+1} \text{ times} \\
 & \longrightarrow D_M(\text{c1notickX}, - \Rightarrow \text{cont}, \\
 & \quad \prod_{h \in 1..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^{\ell+1} \text{ times} \\
 & \longrightarrow D_M(\text{cont}, -, \\
 & \quad \prod_{h \in 1..7} E'_h | E'_X | \{v_1-2, v_2\}^D \downarrow | (V_1^{+5})^{\ell+1} \text{ times} \\
 &) // \text{ repeated } v_1 - 1 \text{ times; } X \text{ takes the value of } X'
 \end{aligned}$$

At this point, the events $\text{dec}_1 \Rightarrow \text{ackdec}_1$ have been moved 5 ticks ahead. We need to do the same for the events $\text{dec}_2 \Rightarrow \text{ackdec}_2$. This will require to advance of two ticks:

$$\begin{aligned}
 & D_M(\text{cont}, -, \prod_{h \in 1..7} E'_h | E'_X | \{0, v_2\}^D \downarrow | (V_1^{+5})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{cont}, - \Rightarrow \text{dec}_1, \\
 & \quad \prod_{h \in 2..7} E'_h | E'_X | \{0, v_2\}^D \downarrow | (V_1^{+5})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_1, -, \prod_{h \in 2..7} E'_h | E'_X | \{0, v_2\}^D \downarrow | (V_1^{+5})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_1, -, (\prod_{h \in 3..7} E'_h) \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+4})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_1, - \Rightarrow \text{dec}_2, \\
 & \quad \prod_{h \in 4..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+4})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_2, -, \prod_{h \in 4..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+4})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_2, -, (\prod_{h \in 4..7} E'_h) \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+3})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_2, - \Rightarrow \text{ackdec}_2, \\
 & \quad \prod_{h \in 5..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+3})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{ackdec}_2, -, \\
 & \quad \prod_{h \in 5..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+3})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{ackdec}_2, - \Rightarrow \text{copy}_2, \\
 & \quad \prod_{h \in 6..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+3})^{v_1-1} \text{ times} \\
 & \longrightarrow D_M(\text{copy}_2, -, \prod_{h \in 6..7} E'_h \downarrow | \{0, v_2\}^D \downarrow | (V_1^{+3})^{v_1-1} \text{ times} \\
 & \longrightarrow // \text{ similar to the above iteration; use fXcopy}_2 \\
 & \longrightarrow D_M(\text{dec}_2, -, \prod_{h \in 6..7} E'_h \downarrow | (V_1^{+3})^{v_1-1} \text{ times} | (V_2^{+5})^{v_2} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_2, -, \\
 & \quad \prod_{h \in 6..7} E'_h \downarrow | (V_1^{+1})^{v_1-1} \text{ times} | (V_2^{+3})^{v_2} \text{ times} \\
 & \longrightarrow D_M(\text{dec}_2, - \Rightarrow Q_j, \{v_1-1, v_2\}^D) \\
 & \longrightarrow D_M(Q_j, -, \{v_1-1, v_2\}^D)
 \end{aligned}$$

where the [TICK] transitions are the unique possible in the corresponding configurations.

$R_1 = 0$: therefore M performs the transition $(Q_i, 0, v_2) \longrightarrow_M (Q_j, 0, v_2)$. Correspondingly, in D_M , an invocation of the

function fzeroQ_i occurs. Let

$$\begin{aligned}
 E_1 &= 0 \gg \text{cont} \Rightarrow \text{dec}_1 \\
 E_2 &= 2 \gg \text{dec}_1 \Rightarrow \text{dec}_2 \\
 E_3 &= 3 \gg \text{ackdec}_2 \Rightarrow \text{copy}_2 \\
 E_4 &= 3 \gg \text{c2notickA} \Rightarrow \text{cont} \\
 E_5 &= 5 \gg \text{dec}_2 \Rightarrow Q_j \\
 E_6 &= 5 \gg \text{notickB} \Rightarrow \text{cont} .
 \end{aligned}$$

Then the contract performs the transitions:

$$\begin{aligned}
 & D_M(Q_i, -, \{0, v_2\}^D) \\
 & \xrightarrow{\text{fAzeroQ}_i} D_M(Q_i, \prod_{h \in 1..6} E_h \Rightarrow \text{notickA}, \{0, v_2\}^D) \\
 & \longrightarrow D_M(Q_i, - \Rightarrow \text{notickA}, \{0, v_2\}^D | \prod_{h \in 1..6} E_h) \\
 & \longrightarrow D_M(\text{notickA}, -, \{0, v_2\}^D | \prod_{h \in 1..6} E_h) \\
 & \longrightarrow D_M(\text{notickA}, -, \{0, v_2\}^D | \prod_{h \in 1..6} E_h) \\
 & \longrightarrow D_M(\text{notickA}, - \Rightarrow \text{cont}, \{0, v_2\}^D | \prod_{h \in 1..6} E_h) \\
 & \longrightarrow D_M(\text{cont}, -, \{0, v_2\}^D | \prod_{h \in 1..6} E_h) \\
 & \longrightarrow D_M(\text{cont}, - \Rightarrow \text{dec}_1, \{0, v_2\}^D | \prod_{h \in 2..6} E_h) \\
 & \longrightarrow D_M(\text{dec}_1, -, \{0, v_2\}^D | \prod_{h \in 2..6} E_h)
 \end{aligned}$$

In this configuration, two tick transitions occur and we have

$$\begin{aligned}
 & D_M(\text{dec}_1, -, \{0, v_2\}^D | \prod_{h \in 2..6} E_h) \\
 & \xrightarrow{2} D_M(\text{dec}_1, -, \{0, v_2\}^D \downarrow | \prod_{h \in 2..6} E_h \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{dec}_1, - \Rightarrow \text{dec}_2, \{0, v_2\}^D \downarrow | \prod_{h \in 3..6} E_h \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{dec}_2, -, \{0, v_2\}^D \downarrow | \prod_{h \in 3..6} E_h \downarrow \downarrow) \\
 & \xrightarrow{2} D_M(\text{dec}_2, -, \{0, v_2\}^D \downarrow \downarrow | \prod_{h \in 3..6} E_h \downarrow \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{dec}_2, - \Rightarrow \text{ackdec}_2, \{0, v_2-1\}^D \downarrow \downarrow | \prod_{h \in 3..6} E_h \downarrow \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{ackdec}_2, -, \{0, v_2-1\}^D \downarrow \downarrow | \prod_{h \in 3..6} E_h \downarrow \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{ackdec}_2, - \Rightarrow \text{copy}_2, \{0, v_2-1\}^D \downarrow \downarrow | \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow) \\
 & \longrightarrow D_M(\text{copy}_2, -, \{0, v_2-1\}^D \downarrow \downarrow | \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow)
 \end{aligned}$$

At this point two transitions may occur: either a tick transition or the management function fAcopy_2 of Table 5 can be invoked. In the first case the contract transits to $D_M(\text{copy}_2, -, \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \downarrow)$ and it is easy to verify that, from this configuration, it is not possible to transit to a configuration $D_M(Q, \Sigma, \Psi)$ anymore. In the second case, by calling fAcopy_2 and fBcopy_2 one after the other for v_2 times (in total), it is possible to move ahead of 5 time units the events $\text{dec}_2 \Rightarrow \text{ackdec}_2$. Let $\prod_{h \in 1..3} E'_h$ where

$$\begin{aligned}
 E'_1 &= 0 \gg \text{cont} \Rightarrow \text{dec}_2 \\
 E'_2 &= 0 \gg \text{ackdec}_2 \Rightarrow \text{copy}_2 \\
 E'_3 &= 0 \gg \text{c2notickB} \Rightarrow \text{cont}
 \end{aligned}$$

and let $V_2^{+k} = k \gg \text{dec}_2 \Rightarrow \text{ackdec}_2$ (as before). Then

$$\begin{aligned}
& D_M(\text{copy}_2, -, \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow) \\
& \xrightarrow{\text{fBcopy}_2} D_M(\text{copy}_2, (\prod_{h \in 1..3} E'_h \mid V_2^{+5}) \Rightarrow \text{c2notickA}, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow) \\
& \rightarrow D_M(\text{copy}_2, - \Rightarrow \text{c2notickA}, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 1..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{c2notickA}, -, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 4..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 1..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{c2notickA}, - \Rightarrow \text{cont}, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 1..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{cont}, -, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 1..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{cont}, - \Rightarrow \text{dec}_2, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 2..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{dec}_2, -, \\
& \quad \{0, v_2 - 1\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 2..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{dec}_2, - \Rightarrow \text{ackdec}_2, \\
& \quad \{0, v_2 - 2\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 2..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{ackdec}_2, -, \\
& \quad \{0, v_2 - 2\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid \prod_{h \in 2..3} E'_h \mid V_2^{+5}) \\
& \rightarrow D_M(\text{ackdec}_2, - \Rightarrow \text{copy}_2, \\
& \quad \{0, v_2 - 2\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid E'_3 \mid V_2^{+5}) \\
& \rightarrow D_M(\text{copy}_2, -, \{0, v_2 - 2\}_A^D \downarrow \downarrow \downarrow \mid \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid E'_3 \mid V_2^{+5}) \\
& \xrightarrow{\text{fBcopy}_2} \dots // \text{ as before } v_2 - 2 \text{ times} \\
& \dots \\
& \rightarrow D_M(\text{dec}_2, -, \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \mid E'_3 \mid (V_2^{+5})_{v_2 \text{ times}}) \\
& \xrightarrow{2} D_M(\text{dec}_2, -, \prod_{h \in 5..6} E_h \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \mid (V_2^{+3})_{v_2 \text{ times}}) \\
& \rightarrow D_M(\text{dec}_2, - \Rightarrow Q_j, \{0, v_2\}_X^D) \\
& \rightarrow D_M(Q_j, -, \{0, v_2\}_X^D)
\end{aligned}$$

where, as before, the last two [TICK] transitions are the unique possible in the corresponding configuration.

As regards (2), we assume that $D_M(Q_i, -, \{v_1, v_2\}_X^D) \xrightarrow{*} D_M(S, \Sigma, \Psi) \rightarrow D_M(Q_j, \Sigma', \Psi')$ is such that $D_M(Q_i, -, \{v_1, v_2\}_X^D) \xrightarrow{*} D_M(S, \Sigma, \Psi)$ does not contain pairs of transitions $D_M(S'', \Sigma'', \Psi'') \rightarrow D_M(Q'', \Sigma'', \Psi'')$ such that $S'' \notin \{Q_0, \dots, Q_n\}$ and $Q'' \in \{Q_0, \dots, Q_n\}$. The general case follows by recurrence.

The proof is a case analysis on the first transition of $D_M(Q_i, -, \{v_1, v_2\}_X^D)$:

- if $D_M(Q_i, -, \{v_1, v_2\}_X^D) \rightarrow D_M(Q_i, -, \{v_1, v_2\}_X^D)$ is an instance of [TICK]. This case is vacuous: the continuations of $D_M(Q_i, -, \{v_1, v_2\}_X^D)$ may be either instances of [TICK] or the invocation of exactly one function $f \in \{\text{fXincQ}_i, \text{fXdecQ}_i, \text{fXzeroQ}_i\}$, where X is either A or B. In this last case, the continuations may only be [TICK] transitions because they transit to states notickX and produce events notickY with $X \neq Y$. Therefore no transition $D_M(S, \Sigma, \Psi) \rightarrow D_M(Q_j, \Sigma', \Psi')$ will be ever possible.
- if $D_M(Q_i, -, \{v_1, v_2\}_X^D) \xrightarrow{\text{fincYQ}_i} D_M(Q_i, Ev \Rightarrow \text{notickY}, \{v_1, v_2\}_X^D)$ then, according to Table 4, M contains the instruction $Q_i : \text{Inc}(R_1, Q_j)$ or $Q_i : \text{Inc}(R_2, Q_j)$. We consider the first case, i.e., R_1 is incremented (the argument for R_2 is similar).

There are two subcases: (i) $X = Y$ and (ii) $X \neq Y$. In case (i), it is easy to verify that $D_M(Q_i, Ev \Rightarrow \text{notickX}, \{v_1, v_2\}_X^D) \xrightarrow{6} D_M(Q_j, -, \{v_1 + 1, v_2\}_Y^D)$ with a deterministic computation that does not traverse configurations $D_M(S'', \Sigma'', \Psi'')$ with $S'' \in \{Q_0, \dots, Q_n\}$. In case (ii), $D_M(Q_i, Ev \Rightarrow \text{notickY}, \{v_1, v_2\}_X^D) \xrightarrow{2} D_M(\text{notickY}, -, \{v_1, v_2\}_X^D | Ev)$. In $\{v_1, v_2\}_X^D | Ev$ there is no event that starts at notickY (and there is no function that can be invoked), therefore the unique possible transitions are [TICK] and no state in $\{Q_0, \dots, Q_n\}$ will be ever reached again – the subcase is vacuous.

- if $D_M(Q_i, -, \{v_1, v_2\}_X^D) \xrightarrow{\text{fdecQ}_i} D_M(Q_i, Ev \Rightarrow \text{notickY}, \{v_1, v_2\}_X^D)$ then, according to Table 4, M contains either $Q_i : \text{DecJump}(R_1, Q', Q')$ or $Q_i : \text{DecJump}(R_2, Q', Q')$. We consider the first case, i.e., R_1 is decreased/tested (the argument for R_2 is similar). There are three subcases:

Subcase $X = Y$ and $v_1 > 0$: Then, letting $E_1 = 1 \gg \text{ackdec}_1 \Rightarrow Q_{j_start1}$ and $E_2 = 1 \gg \text{slnotick} \Rightarrow \text{cont}$, we have

$$\begin{aligned}
& D_M(Q_i, Ev \Rightarrow \text{notickX}, \{v_1, v_2\}_X^D) \\
& \xrightarrow{6} D_M(\text{dec}_1, -, \{v_1, v_2\}_X^D | E_1 | E_2) \\
& \rightarrow D_M(\text{dec}_1, -, \{v_1, v_2\}_X^D \downarrow | E_1 \downarrow | E_2 \downarrow) \\
& \xrightarrow{4} D_M(Q_{j_start1}, -, \{v_1 - 1, v_2\}_X^D \downarrow | E_2 \downarrow)
\end{aligned}$$

where the [TICK] transition in the second line is the unique that is possible. In the state Q_{j_start1} there are two further subcases: (i) a [TICK] transition occurs or (ii) the function fQ_{j_start1} is invoked. The subcase (i) is vacuous because we obtain the configuration $D_M(Q_{j_start1}, -, \{0, v_2\}_X^D \downarrow \downarrow)$ where one may either invoke fQ_{j_start1} or perform [TICK] transitions. In any case, missing an event that starts at slnotick like E_1 , only [TICK] transitions are possible and a state in $\{Q_0, \dots, Q_n\}$ will be never reached. In case (ii), one starts the protocol of Table 5. This protocol transfers the events $1 \gg \text{dec}_1 \Rightarrow \text{ackdec}_1$ and $3 \gg \text{dec}_2 \Rightarrow \text{ackdec}_2$ to 5 ticks ahead by invoking the functions fXcopy_1 and fXcopy_2 , respectively. In correspondence of these invocations, a [TICK] may occurs and the case becomes vacuous (as before); otherwise the protocol continues. No state in $\{Q_0, \dots, Q_n\}$ is ever traversed – no such state is present in the functions fXcopy_1 and fXcopy_2 . When the transfer is completed (and only three [TICK] transitions have been performed), the event $0 \gg \text{dec}_2 \Rightarrow Q_j$ can be executed. The correctness of the final state follows by the discussion in the property (1).

Subcase $X = Y$ and $v_1 = 0$: This case is vacuous because we have

$$\begin{aligned}
& D_M(Q_i, Ev \Rightarrow \text{notickX}, \{0, v_2\}_X^D) \\
& \xrightarrow{6} D_M(\text{dec}_1, -, \{0, v_2\}_X^D | E_1 | E_2)
\end{aligned}$$

where $E_I = 1 \gg \text{ackdec}_1 \Rightarrow Q_{j_start1}$ and $E_2 = 1 \gg \text{slnotick} \Rightarrow \text{cont}$. However, missing any event starting at dec_1 , the contract may only perform $[\text{TICK}]$ transitions.

Subcase $X \neq Y$: This case is also vacuous because

$$\begin{aligned} & D_M(Q_i, Ev \Rightarrow \text{notick}X, \llbracket 0, v_2 \rrbracket_X^D) \\ & \longrightarrow^2 D_M(\text{notick}Y, _, \llbracket 0, v_2 \rrbracket_X^D | Ev) \end{aligned}$$

no event that starts at $\text{notick}Y$ is available. Therefore only $[\text{TICK}]$ transitions can occur and no state in $\{Q_0, \dots, Q_n\}$ can be ever reached.

- if $D_M(Q_i, _, \llbracket v_1, v_2 \rrbracket_X^D) \xrightarrow{\text{fYzero}Q_i} D_M(Q_i, Ev \Rightarrow \text{notick}Y, \llbracket v_1, v_2 \rrbracket_X^D)$ then, according to Table 4, M contains $Q_i : \text{DecJump}(R_1, Q', Q'')$ or $Q_i : \text{DecJump}(R_2, Q', Q'')$. We omit the details of the proof because it is similar to the above case; here the unique non-vacuous subcase is when $X = Y$ and $v_i = 0$.

This concludes the proof. $\square$

B.2 Proofs of Sect. 4

Proposition 1

Let $C(Q, \Sigma, \Psi)$ be a configuration of a $\mu\text{Stipula}^{DI}$ contract C (or a $\mu\text{Stipula}_+^{DI}$ contract). Then

- (i) whenever $Q \notin \text{InitEv}(C)$ or $\Sigma \neq _$:

$$C(Q, \Sigma, \Psi) \longrightarrow \mathbb{C} \text{ if and only if } C(Q, \Sigma, \Psi) \longrightarrow_{tp} \mathbb{C};$$

- (ii) whenever $Q \in \text{InitEv}(C)$ and $\Psi = \mathbf{0} \gg_n Q \Rightarrow Q' \mid \Psi'$:

$$C(Q, _, \Psi) \longrightarrow \mathbb{C} \text{ if and only if } C(Q, _, \Psi) \longrightarrow_{tp} \mathbb{C};$$

- (iii) whenever $Q \in \text{InitEv}(C)$ and $\Psi, Q \nrightarrow$:

$$C(Q, _, \Psi) \text{ is stuck if and only if } C(Q, _, \Psi) \nrightarrow_{tp}.$$

Proof

The proofs of statements (i) and (ii) are case analyses on the possible transition rules that can be applied. They are straightforward and omitted. As regards (iii), we notice that no rule $[\text{FUNCTION}]$ can be applied because $Q \in \text{InitEv}(C)$, hence Q cannot be an initial state of function in $\mu\text{Stipula}^{DI}$ and $\mu\text{Stipula}_+^{DI}$. $[\text{STATE-CHANGE}]$ cannot be applied because $\Sigma = _$ and $[\text{EVENT-MATCH}]$ cannot be applied because $\Psi, Q \nrightarrow$. Therefore, the unique rule that can be applied is $[\text{TICK}]$ and we have

$$C(Q, _, \Psi) \longrightarrow C(Q, _, _) \longrightarrow^* C(Q, _, _)$$

where $C(Q, _, _) \longrightarrow^* C(Q, _, _)$ are instances of $[\text{TICK}]$, therefore $C(Q, _, \Psi)$ is stuck. Correspondingly, since $[\text{TICK}]$ is replaced by $[\text{TICK-PLUS}]$ in $\mu\text{Stipula}_+^{DI}$, no transition $\longrightarrow_{tp}$ is possible. Hence $C(Q, \Sigma, \Psi) \nrightarrow_{tp}$. $\square$

Lemma 1

For a $\mu\text{Stipula}_+^{DI}$ contract C , $(C, \longrightarrow_{tp}, \preceq)$ is a well-structured transition system.

Proof

To verify that $(C, \longrightarrow_{tp}, \preceq)$ is well-structured we need to demonstrate the properties (1) and (2) of Definition 4:

(1) $\preceq$ is a well-quasi ordering. Ψ are multisets of events and the relation $\preceq$ is multiset inclusion. Since the set of all possible events is always finite, by Dickson's Lemma [15], $\preceq$ is a well-quasi ordering.

(2) $\preceq$ is upward compatible with $\longrightarrow_{tp}$. We demonstrate that, if $C(Q, \Sigma, \Psi) \longrightarrow_{tp} C(Q', \Sigma', \Psi')$ and $C(Q, \Sigma, \Psi) \preceq C(Q, \Sigma, \Psi'')$ then there is Ψ''' s.t. $C(Q, \Sigma, \Psi'') \longrightarrow_{tp} C(Q', \Sigma', \Psi''')$ with $C(Q, \Sigma, \Psi') \preceq C(Q, \Sigma, \Psi''')$. The proof consists of a case analysis on the rule used in the transition $C(Q, \Sigma, \Psi) \longrightarrow_{tp} C(Q', \Sigma', \Psi')$. When the rule is an instance of $[\text{FUNCTION}]$ or $[\text{STATE-CHANGE}]$ the upward compatibility is immediate. If the rule is an instance of $[\text{TICK-PLUS}]$ then $Q \notin \text{InitEv}(C)$. Hence the same rule may be applied to $C(Q, \Sigma, \Psi'')$; in this case $\Psi' = _ = \Psi'''$ (because the events in Ψ and Ψ'' have time expressions 0). When the rule is an instance of $[\text{EVENT-MATCH}]$ then let $0 \gg_n Q \Rightarrow Q''$ be the event in Ψ that has been scheduled. Since $\Psi \preceq \Psi''$, the same event can be scheduled in $C(Q, \Sigma, \Psi'')$. By definition, we have $\Psi' \preceq \Psi'''$. $\square$

Lemma 2

Let $(C, \longrightarrow_{tp}, \preceq)$ be the well-structured transition system of a $\mu\text{Stipula}_+^{DI}$ contract. Then, $\longrightarrow_{tp}$ and $\preceq$ are decidable and there exists an algorithm for computing a finite basis of $\uparrow \text{Pred}(\uparrow \mathbb{C})$ for any $\mathbb{C} \in C$.

Proof

Regarding 1, the algorithm for $\preceq$ is trivial, as well as its decidability. The algorithm for $\longrightarrow_{tp}$ is defined by the operational semantics, henceforth its decidability.

Regarding 2, we know that since $\preceq$ is a well-quasi-ordering the set $\uparrow \mathbb{C}$ has a finite basis. Let $\mathcal{B}$ be such basis, we prove that $\uparrow \text{Pred}(\mathcal{B})$ has a finite basis that is also computable. Let $C' \in \mathcal{B}$; there are three cases:

$\mathbb{C}' = C(Q', \Sigma, \Psi)$ and $\Sigma \neq _$: In this case, the transition $\mathbb{C}'' \longrightarrow_{tp} \mathbb{C}'$ can be obtained in two possible ways: by applying $[\text{FUNCTION}]$ or $[\text{EVENT-MATCH}]$. In the first case, $\text{Pred}(\mathcal{B})$ contains all the tuples $C(Q', _, \Psi)$ for which there is a function $@Q' \text{ f } \{W\} \Rightarrow @Q'' \in C$ that can produce Σ ; in the second case, $\text{Pred}(\mathcal{B})$ contains all the tuples $C(Q', _, \Psi')$ where Ψ' extends Ψ with an event $0 \gg @Q' \Rightarrow @Q'' \in C$ that can produce Σ . In both cases, $\text{Pred}(\mathcal{B})$ is finite and computable.

$C' = C(Q', _, \Psi)$ and $\Psi \neq _$: The unique rule that gives a transition $C'' \rightarrow_{\text{tp}} C'$ is [STATE-CHANGE]. In this case we have $C'' = C(Q, W \Rightarrow Q', \Psi')$ where $W = (\emptyset \gg_{n_i} Q_i \Rightarrow Q'_i)^{i \in 1..h}$ and $\Psi = \Psi' | \emptyset \gg_{n_1} Q_1 \Rightarrow Q'_1 | \dots | \emptyset \gg_{n_h} Q_h \Rightarrow Q'_h$. There are two possible cases for obtaining the terms $W \Rightarrow Q'$: (i) the functions $@Q \not\equiv \{W\} \Rightarrow @Q' \in C$, for every possible subterm W of Ψ ; and (ii) the events $\emptyset \gg @Q \Rightarrow @Q' \in C$ (in this subcase, $W = _$). In both cases, $\text{Pred}(\mathcal{B})$ is finite and computable.

$C' = C(Q', _, _)$: Then the transition $C'' \rightarrow_{\text{tp}} C'$ is an instance of [TICK-PLUS]. C'' could be any tuple of type $C(Q', _, \Psi')$. $\{C(Q', _, _)\}$ is a finite basis for all such tuples. $\square$

Funding information Open access funding provided by Università degli Studi di Genova within the CRUI-CARE Agreement.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D.: Compilers: Principles, Techniques, and Tools, 2nd edn. Addison Wesley, Boston (2006)
- Alur, R., Dill, D.L.: A theory of timed automata. *Theor. Comput. Sci.* **126**(2), 183–235 (1994)
- Amadio, R.M., Meyssonier, C.: On decidability of the control reachability problem in the asynchronous pi-calculus. *Nord. J. Comput.* **9**(2), 70–101 (2002)
- Appel, A.W., Ginsburg, M.: Modern Compiler Implementation in C. Cambridge University Press, Cambridge (1997)
- Abdulla, P.A., Jonsson, B.: Verifying programs with unreliable channels. In: Proceedings of the Eighth Annual Symposium on Logic in Computer Science (LICS'93), Montreal, Canada, June 19–23, 1993, pp. 160–170. IEEE Comput. Soc., Los Alamitos (1993)
- Abdulla, P.A., Jonsson, B.: Model checking of systems with many identical timed processes. *Theor. Comput. Sci.* **290**(1), 241–264 (2003)
- Abdulla, P.A., Nylén, A.: Timed Petri nets and bqos. In: ICATPN'01. LNCS, vol. 2075, pp. 53–70. Springer, Berlin (2001)
- Abdulla, P.A., Cerans, K., Jonsson, B., Tsay, Y.-K.: General decidability theorems for infinite-state systems. In: Proc. 11th Annual IEEE Symposium on Logic in Computer Science, pp. 313–321. IEEE Comput. Soc., Los Alamitos (1996)
- Abdulla, P.A., Delzanno, G., Rezine, O., Sangnier, A., Traverso, R.: Parameterized verification of time-sensitive models of ad hoc network protocols. *Theor. Comput. Sci.* **612**, 1–22 (2016)
- Cécé, G., Finkel, A., Purushothaman Iyer, S.: Unreliable channels are easier to verify than perfect channels. *Inf. Comput.* **124**(1), 20–31 (1996)
- Crafa, S., Laneve, C.: Programming legal contracts - a beginners guide to Stipula. In: The Logic of Software. A Tasting Menu of Formal Methods - Essays Dedicated to Reiner Hähnle on the Occasion of His 60th Birthday. Lecture Notes in Computer Science, vol. 13360, pp. 129–146. Springer, Cham (2022)
- Crafa, S., Laneve, C., Sartor, G., Veschetti, A.: Pacta sunt servanda: legal contracts in Stipula. *Sci. Comput. Program.* **225**, 102911 (2023)
- de Boer, F.S., Mahdi Jaghoori, M., Laneve, C., Zavattaro, G.: Decidability problems for actor systems. *Log. Methods Comput. Sci.* **10**(4) (2014)
- Delzanno, G., Laneve, C., Sangnier, A., Zavattaro, G.: Decidability problems for micro-stipula. In: Di Giusto, C. and Ravara, A. (eds.), Coordination Models and Languages - 27th IFIP WG 6.1 International Conference, COORDINATION 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025, Proceedings, Lille, France, June 17–19, 2025. Lecture Notes in Computer Science, vol. 15731, pp. 133–152. Springer, Berlin (2025)
- Dickson, L.E.: Finiteness of the odd perfect and primitive abundant numbers with $5n$ distinct prime factors. *Bull. Am. Math. Soc.* **19**(6), 285 (1913)
- Dufourd, C., Finkel, A., Schnoebelen, P.: Reset nets between decidability and undecidability. In: Larsen, K.G., Skyum, S., Winskel, G. (eds.) Automata, Languages and Programming, 25th International Colloquium, ICALP'98, Aalborg, Denmark, July 13–17, 1998. Lecture Notes in Computer Science, vol. 1443, pp. 103–115. Springer, Berlin (1998)
- Esparza, J., Nielsen, M.: Decidability issues for Petri nets - a survey. *J. Inf. Process. Cybern.* **30**(3), 143–160 (1994)
- Finkel, A., Schnoebelen, Ph.: Well-structured transition systems everywhere! *Theor. Comput. Sci.* **256**, 63–92 (2001)
- Finkel, A., Raskin, J.-F., Samuelides, M., Van Begin, L.: Monotonic extensions of Petri nets: forward and backward search revisited. *Electron. Notes Theor. Comput. Sci.* **68**(6), 85–106 (2002)
- Ganty, P., Majumdar, R.: Algorithmic verification of asynchronous programs. *ACM Trans. Program. Lang. Syst.* **34**(1), 6:1–6:48 (2012)
- Hansson, H., Jonsson, B.: A calculus for communicating systems with time and probabilities. In: Proceedings of the Real-Time Systems Symposium - 1990, pp. 278–287. IEEE Comput. Soc., New York (1990)
- Jhala, R., Majumdar, R.: Interprocedural analysis of asynchronous programs. In: Hofmann, M., Felleisen, M. (eds.) Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17–19, 2007, pp. 339–350. ACM, New York (2007)
- Karp, R.M., Miller, R.E.: Parallel program schemata. *J. Comput. Syst. Sci.* **3**(2), 147–195 (1969)
- Laneve, C.: Reachability analysis in micro-Stipula. In: Proc. 26th International Symposium on Principles and Practice of Declarative Programming, PPDP 2024, pp. 17:1–17:12. ACM, New York (2024)
- Mayr, R.: Undecidability of weak bisimulation equivalence for 1-counter processes. In: Baeten, J.C.M., Karel Lenstra, J., Parrow, J., Woeginger, G.J. (eds.) Automata, Languages and Programming, 30th International Colloquium, ICALP 2003, Proceedings, Eindhoven, The Netherlands, June 30–July 4, 2003. Lecture Notes in Computer Science, vol. 2719, pp. 570–583. Springer, Berlin (2003)
- Meyer, R.: On boundedness in depth in the pi-calculus. In: IFIP TCS. IFIP, vol. 273, pp. 477–489. Springer, Berlin (2008)
- Minsky, M.: Computation: Finite and Infinite Machines. Prentice Hall, New York (1967)
- Moller, F., Tofts, C.M.N.: A temporal calculus of communicating systems. In: Proceedings of CONCUR'90. Lecture Notes in Computer Science, vol. 458, pp. 401–415. Springer, Berlin (1990)

-
29. Sen, K., Viswanathan, M.: Model checking multithreaded programs with asynchronous atomic methods. In: Ball, T., Jones, R.B. (eds.) *Computer Aided Verification*, 18th International Conference, CAV 2006, Proceedings Seattle, WA, USA, August 17–20, 2006. *Lecture Notes in Computer Science*, vol. 4144, pp. 300–314. Springer, Berlin (2006)
 30. Stipula Reachability Analyzer, S.E.: (2024). Available on github. <https://github.com/stipula-language>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.